Fpt University

TRƯỜNG ĐẠI HỌC FPT

Hotel Management System

Software Design Document

Project Code: HMS

Document Code: HMS_SDD– v1.0

– Ha Noi, October, 2026 –

Table of Contents

1 Introduction

1.1 Purpose

1.2 Definitions, Acronyms and Abbreviations

1.3 References

2 System Architecture

2.1 System Overview

2.2 Assumptions

2.3 Design Constraints

3 Software Architecture Design

3.1 Architectural Representation

3.2 Software Architecture

3.2.1 Process View

3.2.2 Logical View

3.2.3 Development View

3.2.4 Deployment View

4 Detailed Component Design

4.1 Common Design

4.1.1 Exception Handling

4.1.2 Logging (Audit Logging)

4.1.3 Messages Processing

4.1.4 UI Structures

4.1.5 Utility Classes/Components

4.1.6 Transaction Management

4.2 Account & Authentication Management

4.2.1 Screen Design

4.2.2 Class/Component Design

4.2.3 Object Interactions (Sequence Diagram)

4.3 Room & Facility Management

4.3.1 Screen Design

4.3.2 Class/Component Design

4.3.3 Object Interactions (Sequence Diagram)

4.4 Reservation & Front-Desk Operations

4.4.1 Screen Design

4.4.2 Class/Component Design

4.4.3 Object Interactions (Sequence Diagram)

4.5 Ancillary Services & Billing Management

4.5.1 Screen Design

4.5.2 Class/Component Design

4.5.3 Object Interactions (Sequence Diagram)

4.6 Reviews, Audit & Reporting Management

4.6.1 Screen Design

4.6.2 Class/Component Design

4.6.3 Object Interactions (Sequence Diagram)

5. Database Design

5.1 Database Design

a. Role

b. User

c. Room_Type

d. Room

e. Reservation

f. Reservation_Detail

g. Guest_Stay

h. Stay_Room_Assignment

i. Room_Maintenance

j. Review

k. Hotel_Service

l. Service_Order

m. Folio

n. Folio_Item

o. Voucher

p. Invoice

q. Payment

r. Customer

5.2 Data File Design

1 Introduction

1.1 Purpose

This Software Design Document (SDD) provides a comprehensive architectural and detailed technical design for the Hotel Management System (HMS). It translates the functional and non-functional requirements specified in the *HMS Software Requirement Specification (HMS-SRS-v1.2)* into an executable software architecture, component structure, and relational database schema.

Specifically, this document contains the following information:

- System Architecture (Section 2): Defines the high-level Client-Server architecture, the interaction between user groups, the frontend Single Page Application (SPA), the backend application server, the relational database server (Microsoft SQL Server), and external integrated services (Payment Gateway and Mail Notification Service), along with system assumptions and design constraints.
- Software Architecture Design (Section 3): Presents the architectural views of the system, including the Process View, Logical View, Development View, and Deployment View, following the 3-Layer Architecture pattern.
- Detailed Component Design (Section 4): Specifies the common design mechanisms (error handling, logging, security filters, transaction management) and the detailed class diagrams, class specifications, screen definitions, and sequence diagrams for the system's features and use cases.
- Database Design (Section 5): Details the physical relational database tables in Microsoft SQL Server and the mapped JPA Entity and Data Transfer Object (DTO) classes.

This document is intended for the following readers:

- SBA301 Instructors: To review and evaluate the architectural soundness, adherence to software engineering standards, and compliance with the SBA301 technical stack.
- Software Developers (Frontend & Backend Engineers): To serve as the authoritative technical specification for implementing ReactJS components, Spring Boot REST controllers, business services, and Spring Data JPA repositories.
- Quality Assurance (QA) / Testing Engineers: To understand component interfaces, API contracts, and transaction boundaries when designing unit and integration test cases.

1.2 Definitions, Acronyms and Abbreviations

The table below lists all common business and technical definitions, acronyms, and abbreviations used throughout this Software Design Document:

Term / Acronym	Full Form / Definition	Description
HMS	Hotel Management System	The centralized web-based software platform designed to coordinate daily lodging operations across independent 3-star to 4-star hotels.
SRS	Software Requirement Specification	The baseline requirements document (HMS-SRS-v1.2) specifying the functional and non-functional requirements of the HMS.
SDD	Software Design Document	This document, which describes the system architecture, component design, and database design of the HMS.
SPA	Single Page Application	A web application implemented in ReactJS that dynamically updates the current page via JavaScript without requiring full page reloads from the server.
CSR	Client-Side Rendering	A rendering model in which the client web browser renders the user interface using JavaScript and exchanges JSON data with backend APIs.
REST	Representational State Transfer	An architectural style for networked applications using standard stateless HTTP methods (GET, POST, PUT, DELETE) to manipulate resources.
API	Application Programming Interface	A set of defined endpoints exposed by the backend server to enable communication with the frontend client and external services.
JWT	JSON Web Token	An open standard (RFC 7519) used for stateless user authentication and

		authorization between the client browser and the backend server.
RBAC	Role-Based Access Control	A security approach that restricts system access to authorized user roles (Customer, Front-Desk Staff, Housekeeping Staff, Service Staff, Administrator).
ORM	Object-Relational Mapping	A programming technique (using Hibernate) that maps object-oriented domain entities in Java to relational database tables.
JPA	Java Persistence API	The Jakarta EE specification used via Spring Data JPA for managing relational data in Java applications.
DTO	Data Transfer Object	An object used to encapsulate and transfer data between the presentation layer and the service layer, decoupling database entities from API payloads.
DAO / Repository	Data Access Object / Repository	The persistence layer abstraction responsible for executing database CRUD queries and encapsulating data access logic.
RDBMS	Relational Database Management System	The relational database engine (Microsoft SQL Server) used to store structured, transactional hotel data.
Folio	Guest Stay Folio	The financial ledger of an active guest stay that accumulates room charges, service orders, and surcharges prior to invoice generation.
SMTP	Simple Mail Transfer Protocol	The standard communication protocol used to send outbound transactional emails via an external mail server.

1.3 References

The following table lists the documents, standards, and technical specifications referenced by this Software Design Document:

No.	Document Title / Standard	Version	Date	Author / Source / Location
1	Hotel Management System – Software Requirement Specification (HMS-SRS)	v1.2	Oct 2026	HMS Project Team (KhangND, HieuVQ, GiapND, LinhNH, TungPV) – FPT University
2	Software Design Document (SDD) Template	v1.0	2026	Department of Software Engineering, FPT University
3	SBA301 Courseware & Lecture Materials	2026	2026	FPT University Academic Curriculum (ReactJS SPA, RESTful APIs, Spring Boot 3, JPA/Hibernate, MongoDB, JWT Security)
4	Spring Boot & Spring Security Reference Documentation	3.2+	2024	VMware / Spring.io (https://docs.spring.io/spring-boot/)
5	React Official Documentation	18.2+	2024	Meta Open Source (https://react.dev/)
6	OpenAPI Specification (Springdoc-OpenAPI / Swagger 3)	3.0.3	2024	OpenAPI Initiative (https://swagger.io/specification/)
7	RFC 7519 – JSON Web Token (JWT)	Standard	May 2015	Internet Engineering Task Force (https://datatracker.ietf.org/doc/html/rfc7519)

2 System Architecture

2.1 System Overview

The Hotel Management System (HMS) is designed using a decoupled Client-Server Architecture. All end-user groups interact with the system through a modern web browser running a Single Page Application (SPA), which communicates with the backend application server over HTTP/HTTPS using RESTful JSON APIs. The backend server processes business logic, enforces security and transaction rules, persists data across relational and document databases, and integrates with external payment and email services.

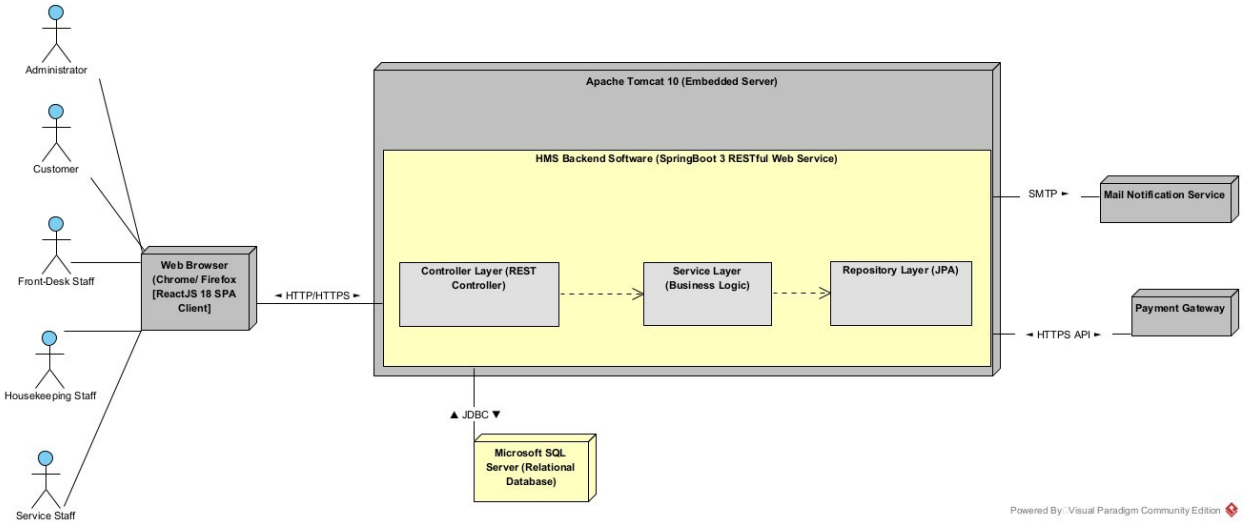


Figure 2-1: System Architectural Overview

As described in Figure 2-1, the system will have the following components:

- User Groups (Actors):
    - Customer: An external guest who accesses public and customer-specific features via a web browser to browse room categories, search real-time room availability, create online reservations with advance deposit payments, order room services during an active stay, view booking histories, and submit room reviews after departure.
    - Front-Desk Staff: A receptionist responsible for front-office operations, including confirming pending offline/direct reservations, verifying guest identification and allocating clean physical rooms at check-in, handling mid-stay room transfers and stay extensions, compiling itemized invoices, applying discount vouchers, recording counter payments, and completing guest check-outs.
    - Housekeeping Staff: A floor staff member or housekeeping supervisor who monitors room inventory, updates room cleanliness states (*Dirty* → *Clean* → *Inspected*), and places out-of-order rooms under maintenance.

- Service Staff: An operational staff member from ancillary service departments (F&B, room service, minibar, spa, laundry) who views active service orders, posts consumed service charges onto active guest stay folios, and voids unrendered service items.
- Administrator: A hotel manager, accountant, or IT administrator who configures master data (room types, physical rooms, hotel services, promotional vouchers), manages user accounts and role assignments, moderates customer reviews, inspects system audit logs, and generates occupancy and revenue analytical reports.

- Web Browser & HMS ReactJS SPA Client: This is the client-side presentation component accessed by all five user groups on desktop or mobile web browsers (Google Chrome, Mozilla Firefox, Microsoft Edge). It is developed as a Single Page Application (SPA) using ReactJS 18, React Router DOM for client-side navigation across the 54 screens defined in the SRS, React-Bootstrap for responsive UI layouts, and Axios for sending asynchronous HTTP/HTTPS requests with JSON payloads and JWT Bearer tokens to the backend server.
- HMS Backend Software: This is the core developing server software built on Java 21 and the Spring Boot 3 framework, deployed on an embedded Apache Tomcat 10 application server. Internally, it is organized into a strict 3-Layer Architecture (Controller → Service → Repository), secured by Spring Security 6 with stateless JWT (JSON Web Token) authentication, and documented using Springdoc-OpenAPI (Swagger 3).
- Microsoft SQL Server 2022: This is the primary Relational Database Management System (RDBMS) connected to the HMS Backend Software via JDBC and managed through Spring Data JPA / Hibernate ORM. It stores all structured, relational, and ACID-compliant operational entities, including user accounts, room types, physical rooms, reservations, guest stays, hotel services, service orders, folios, invoices, vouchers, payments, and customer reviews.
- Payment Gateway (VNPay / OnePay): This is an external third-party electronic payment processing system that the HMS Backend Software integrates with over HTTPS REST APIs to authorize online reservation deposit payments (UC-23) and receive authenticated transaction confirmation results.
- Mail Notification Service: This is an external mail server (e.g., Google SMTP Service) that the HMS Backend Software integrates with via the SMTP protocol (JavaMailSender) in order to send password recovery verification links (UC-10), booking confirmation notices (UC-23, UC-26), and digital itemized invoice receipts to users.

2.2 Assumptions

This system is designed based on the following assumptions regarding hardware, software, operating systems, end-user characteristics, and operational environment:

- Server Operating System & Runtime Environment:
  - Windows Server 2019/2022 or Ubuntu Linux 22.04 LTS (64-bit).

- Java Development Kit (JDK) 21 LTS installed on the application server with a minimum of 8 GB RAM.
- Node.js v18.x LTS or higher and npm v9.x+ available in the build environment for compiling the ReactJS frontend bundle.

- Database Management Systems:
  - Microsoft SQL Server 2019 or 2022 installed and accessible via TCP port 1433.
  - Automated nightly database backups are configured at the infrastructure level to retain backups for 30 days and maintain a Recovery Point Objective (RPO) of less than 24 hours (OR-04).
- Client Hardware, Operating Systems & Software:
  - End-user devices (desktop PCs at the front desk, laptops, tablets, or smartphones) running Windows 10/11, macOS, Android, or iOS.
  - Modern ECMAScript 6+ compliant web browsers: Google Chrome (v110+), Mozilla Firefox (v110+), Microsoft Edge (v110+), or Apple Safari (v16+) with JavaScript and localStorage enabled.
  - Standard A4/A5 laser or thermal receipt printers connected to front-desk workstations and PDF viewer software (e.g., Adobe Acrobat Reader or built-in browser print engine) to print itemized invoice receipts (Section 5.1.2 & OR-02 of SRS).
- Network & External Services Availability:
  - The hotel property (60 to 150 rooms) has stable local area network (LAN) and continuous broadband internet connectivity.
  - External third-party services—specifically the Payment Gateway (VNPay/OnePay) and the SMTP Mail Notification Service—are operational, reachable over the internet, and respond within standard network timeout limits.
- End-User Characteristics & Operational Scope:
  - Hotel staff (Front-Desk, Housekeeping, Service Staff, and Administrators) possess basic computer literacy and web browser navigation skills.
  - Physical room key cards are handed over manually by front-desk receptionists and recorded by serial number in the Guest Check-in Form (UC-27); direct hardware integration with physical RFID door lock encoders is outside the scope of this software

-

2.3 Design Constraints

This system must comply with the following global limitations, architectural standards, and business rules imposed by the HMS-SRS-v1.2 and the SBA301 technical framework:

- Architectural & Technology Stack Constraints:
  - 3-Layer Backend Architecture: The Spring Boot backend must strictly separate responsibilities into three layers: controllers (REST API handling), services (business logic and transaction management), and repositories (data persistence). Controllers must not access Repositories directly, and database Entity objects must not be exposed directly to the client without Data Transfer Object (DTO) mapping.
  - Single Page Application (SPA) Frontend: The user interface must be implemented as a ReactJS SPA using Functional Components and React Hooks (useState, useEffect, useContext). Legacy React Class Components and server-side HTML rendering (e.g., JSP) are prohibited.
  - Client-Server Communication Protocol: All data transfers between the web browser and the backend server must be executed asynchronously via Axios over HTTP/HTTPS using RESTful conventions and application/json; charset=UTF-8 payloads.
- Performance & Scalability Constraints:
  - 3-Second Response Time SLA: In compliance with Section 5.2.3 of the SRS, every core operation—including Room Availability Search (UC-21), Booking Creation (UC-23), Guest Check-in (UC-27), Invoice Compilation (UC-37), Payment Settlement (UC-39), and Report Generation (UC-45..47)—must complete and display results within 3 seconds under normal operating conditions.
  - Server-Side Pagination: All list and search endpoints (room inventory, customer bookings, user accounts, service catalog, and audit logs) must implement server-side pagination and filtering using Spring Data's Pageable and Page<T> interfaces to prevent memory bottlenecks (CR-10).
- Security & Access Control Constraints:
  - Stateless JWT Authentication & Account Status Enforcement: Protected API endpoints must require a valid JWT Bearer token in the Authorization HTTP header. Only accounts with Active status are permitted to access the system (BR-02), and deactivating a user account (UC-08) must immediately revoke access for any active tokens belonging to that account.
  - Role-Based Authorization: Both frontend route guards and backend Spring Security filters must strictly enforce the role permissions defined in the Screen Authorization Matrix (Section 3.1.3 of SRS and CR-06).

- Password Cryptography: User passwords must never be stored or transmitted in plaintext (Section 5.2.4 of SRS); all passwords must be hashed using the one-way BCrypt hashing algorithm before being persisted in the database.

- Data Repository, Transaction & Business Rule Constraints:
  - ACID Transaction Management: Multi-step state changes—such as Guest Check-in (updating Reservation status to Checked-In, creating a Guest Stay, initializing a Folio, and setting Room status to Occupied), Room Transfer (UC-28), Stay Extension (UC-29), and Guest Check-out (UC-30)—must be executed within atomic database transactions (@Transactional) so that any failure rolls back all partial changes (Section 5.2.2 & 5.2.6 of SRS).
  - Configuration Isolation for Historical Records: In accordance with Section 5.2.5 of the SRS and BR-27, updates to room category base tariffs (UC-12) or hotel service unit prices (UC-32) must not alter completed reservations, historical service orders, or finalized invoices. Transactional tables must snapshot the applicable unit price at the time of creation.
  - 24-Hour Pending Reservation Hold: New reservations created in Pending status must hold room category quota for 24 hours and be automatically cancelled by a scheduled background task (@Scheduled) if deposit or confirmation is not completed within 24 hours (BR-19).
  - Immutable Audit Trail & Retention Policy: All monetary operations (deposit, payment, refund – CR-11) and auditable administrative actions must be logged in an append-only audit store, retained for a minimum of 12 months, and restricted from editing or deletion by operational staff (BR-42, OR-01).
  - Currency & Rounding Standard: All monetary amounts must be non-negative, stored using exact decimal types (BigDecimal / DECIMAL), displayed in VND, and rounded to the nearest thousand (CR-08, CR-12).

- User Interface, Localization & API Documentation Constraints:
  - Bilingual Support: The user interface must support both Vietnamese (default for staff roles) and English (default for guests/customers) as specified in OR-05.
  - Standardized Validation & Application Messages: Failed input validations must keep the user on the same screen and return standardized inline message codes (MSG01A to MSG12B) matching Section 6.2 (CR-01 to CR-12) and Section 6.3 of the SRS.
  - OpenAPI / Swagger Compliance: All RESTful API endpoints must be documented using Springdoc-OpenAPI (Swagger 3) annotations to facilitate verification, validation, and API testing.

3 Software Architecture Design

3.1 Architectural Representation

The Hotel Management System (HMS) architecture is represented through four architectural views: Logical View, Process View, Development View, and Deployment View. These views describe the system structure, runtime behavior, source code organization, and physical deployment.

View	Purpose	UML Diagrams Used
Logical View	Describes the internal structure of the HMS, including major classes, responsibilities, and relationships.	Class Diagram
Process View	Describes runtime behavior and interactions among system components during major hotel operations.	Activity Diagram
Development View	Describes the organization of the system into frontend modules, backend layers, and package dependencies.	Package Diagram
Deployment View	Describes how the ReactJS client, Spring Boot backend, SQL Server, MongoDB, and external services are deployed and connected.	Deployment Diagram

3.2 Software Architecture

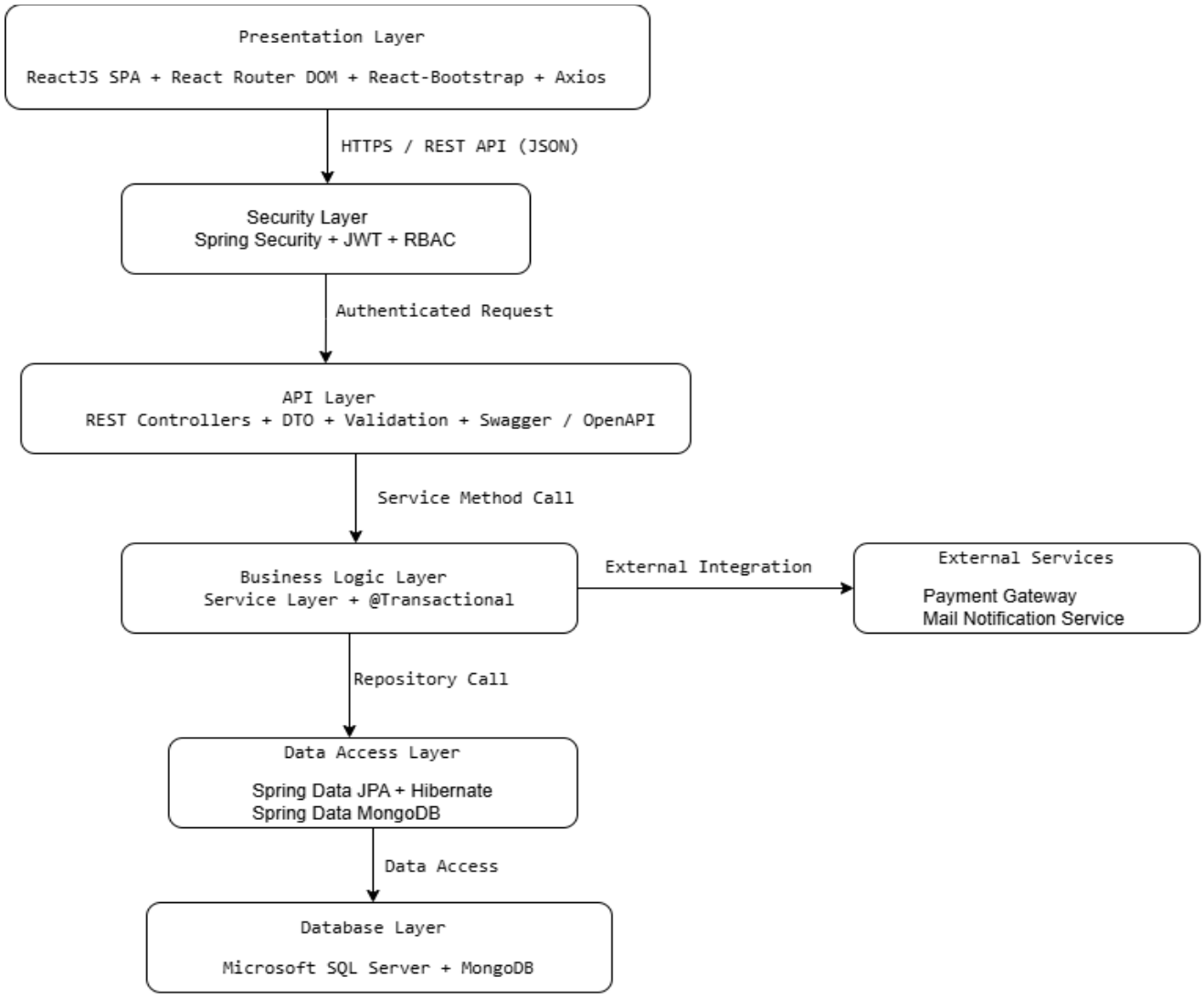


Figure 3-1: HMS Layered Architecture Pattern

Layer	Responsibility
Presentation	Renders the user interface, handles user interactions, manages client-side navigation, and sends HTTP/HTTPS requests via Axios.
Security	Intercepts protected requests, validates JWT tokens, verifies account status, and enforces role-based access control.
API	Receives REST requests, validates input data, delegates processing to the Business Logic Layer, and returns DTO responses.

Business Logic	Implements business rules, manages transactions, coordinates repositories, and integrates with external services.
Data Access	Abstracts persistence operations using Spring Data JPA, Hibernate ORM, and Spring Data MongoDB.
Database	Persists structured transactional data in Microsoft SQL Server and audit log documents in MongoDB.

3.2.1 Process View

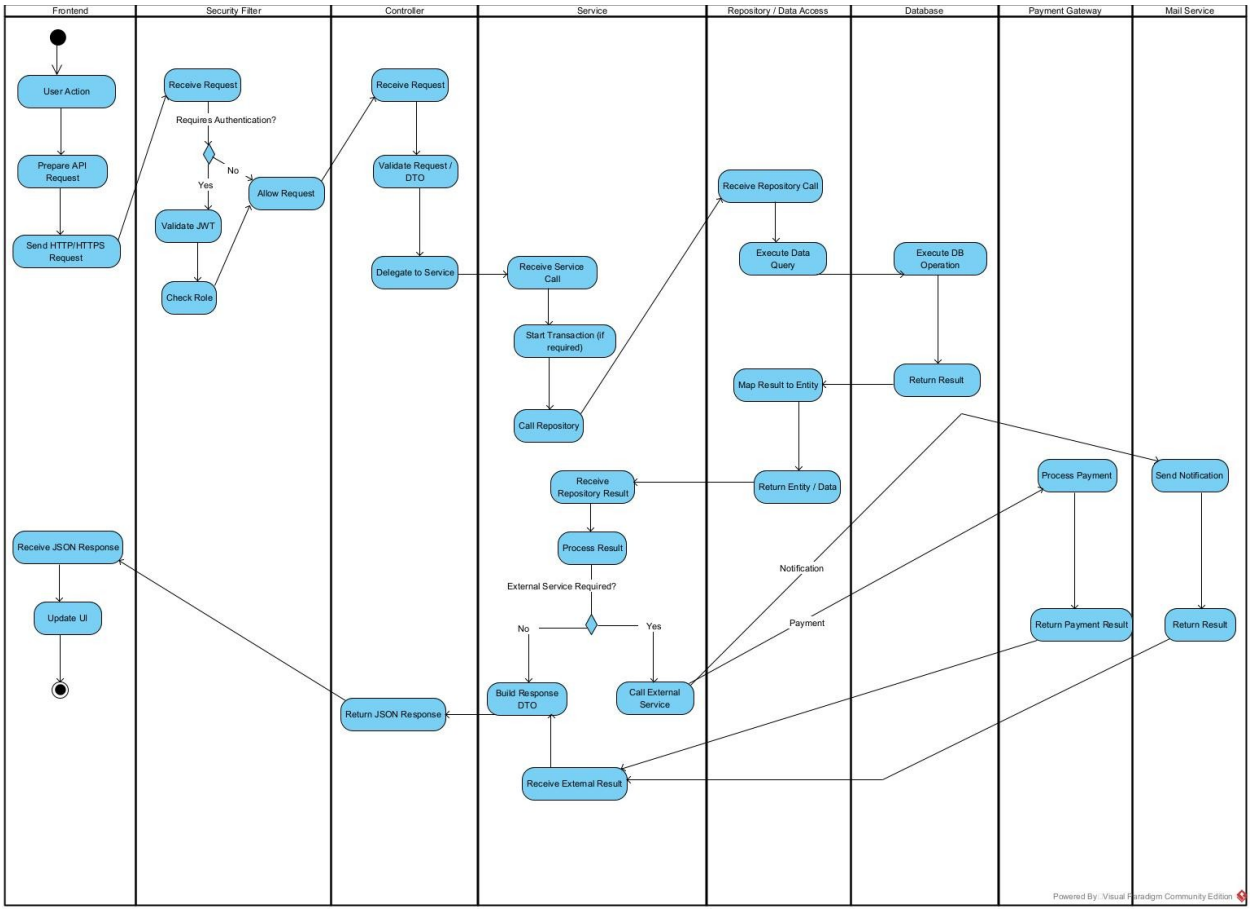


Figure 3-2: HMS Process Overview

3.2.2 Logical View

3.2.2.1 Frontend Logical View

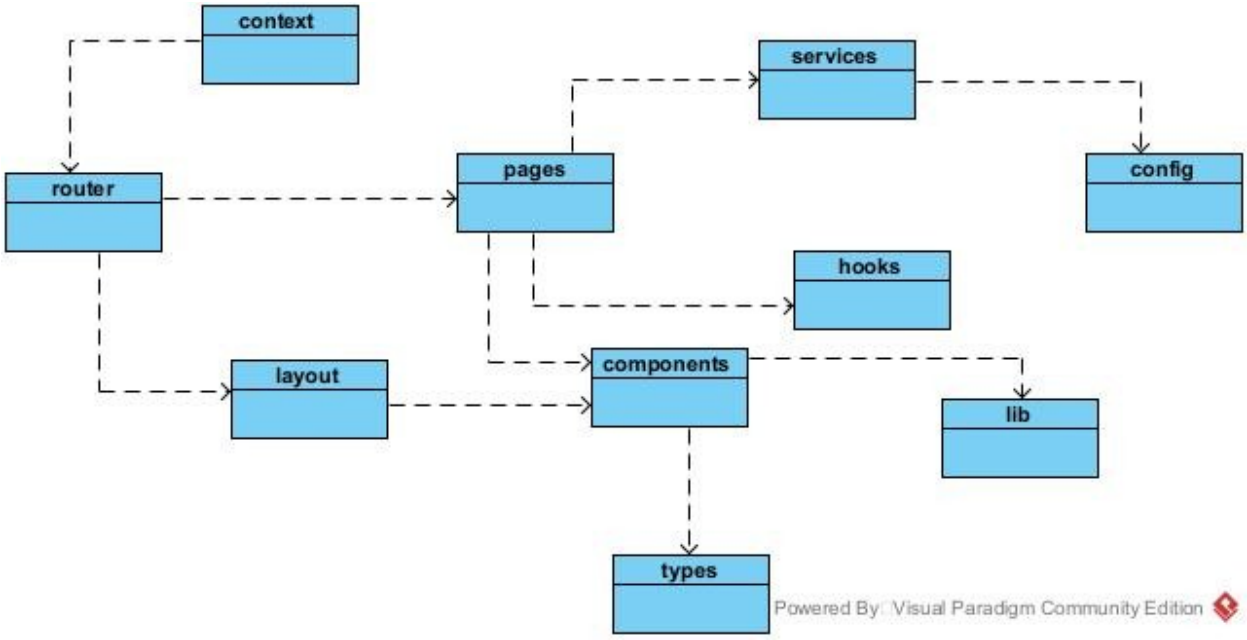


Figure 3-3: HMS FrontEnd Logical Overview

Package	Responsibility
contexts	Provides shared application state such as authentication status, current user information, and role data.
config	Stores application configuration such as API base URLs, constants, and environment settings.
router	Manages application routes, navigation, protected routes, and role-based access using React Router DOM.
layouts	Provides common page layouts such as headers, navigation bars, sidebars, and page containers.
pages	Contains page-level modules for major HMS functions such as booking, check-in, active stay, housekeeping, services, reviews, and administration.

components	Contains reusable UI components shared across multiple pages.
services	Handles communication with backend REST APIs using Axios and organizes API calls by functional module.
hooks	Contains reusable React Hooks for state handling, data loading, and shared frontend logic.
lib	Provides common utility functions and helper logic used by frontend modules.
types	Defines shared data structures and frontend data models used across components and services.

3.2.2.1 Backend Logical View

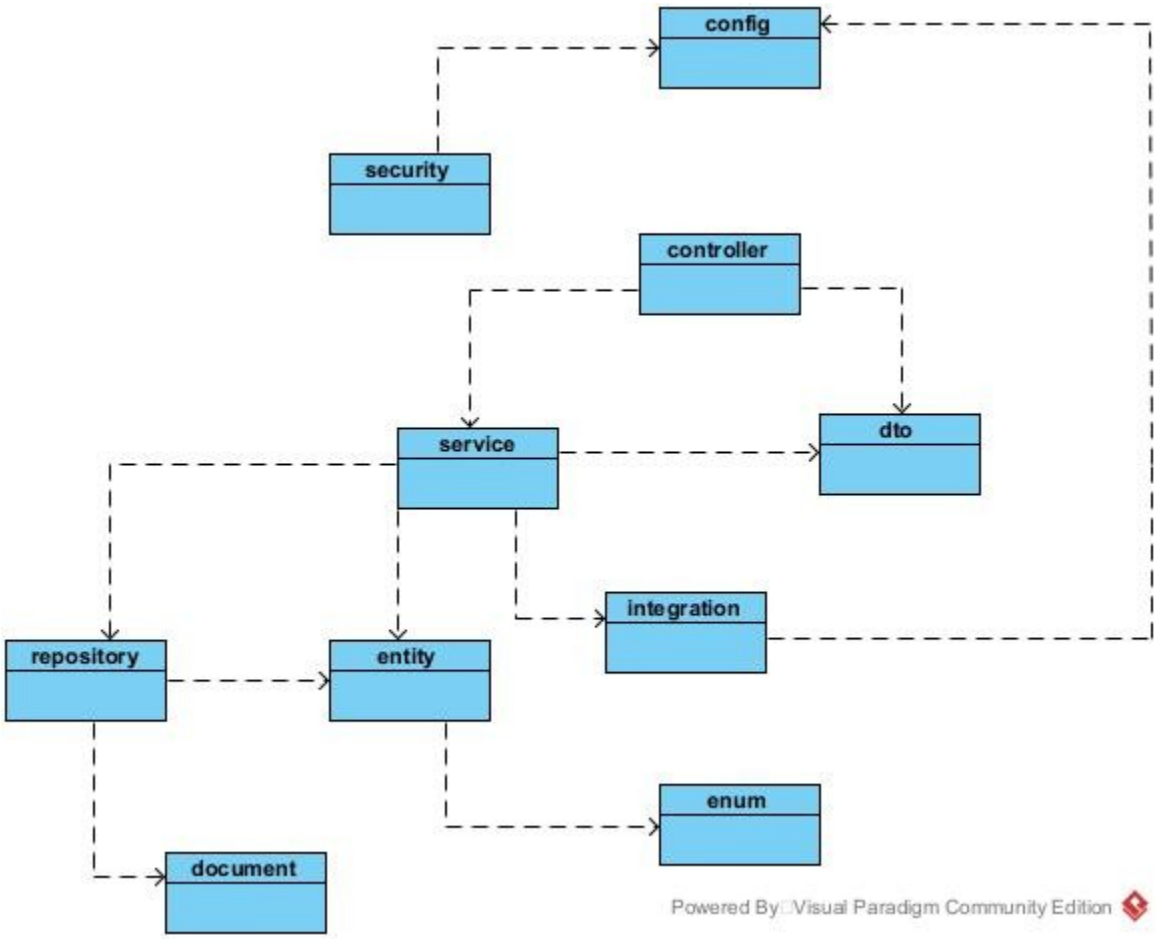


Figure 3-4: HMS Back-End Logical Overview

Package	Responsibility
config	Contains application-level configuration for CORS, Swagger/OpenAPI, database access, security, payment gateway, and mail integration.
security	Handles JWT authentication, password encoding, account validation, and role-based authorization.
controller	Receives REST API requests, validates request data, delegates processing to the Service layer, and returns response DTOs.
service	Implements business rules, manages transactions, coordinates repositories, and invokes external integrations when required.
repository	Provides data access operations for SQL Server and MongoDB using Spring Data repositories.
entity	Contains JPA entity models mapped to relational tables in Microsoft SQL Server through Hibernate ORM.
dto	Defines request and response data transfer objects used between API and service layers.
integration	Handles communication with external systems such as the Payment Gateway and Mail Notification Service.
document	Contains MongoDB document models used for document-based data such as system audit logs.
enum	Defines shared constant value types such as user roles, reservation status,

	room status, payment status, and other domain states.

3.2.3 Development View

3.2.3.1 Package Diagram

3.2.3.1.1 Back end package diagram

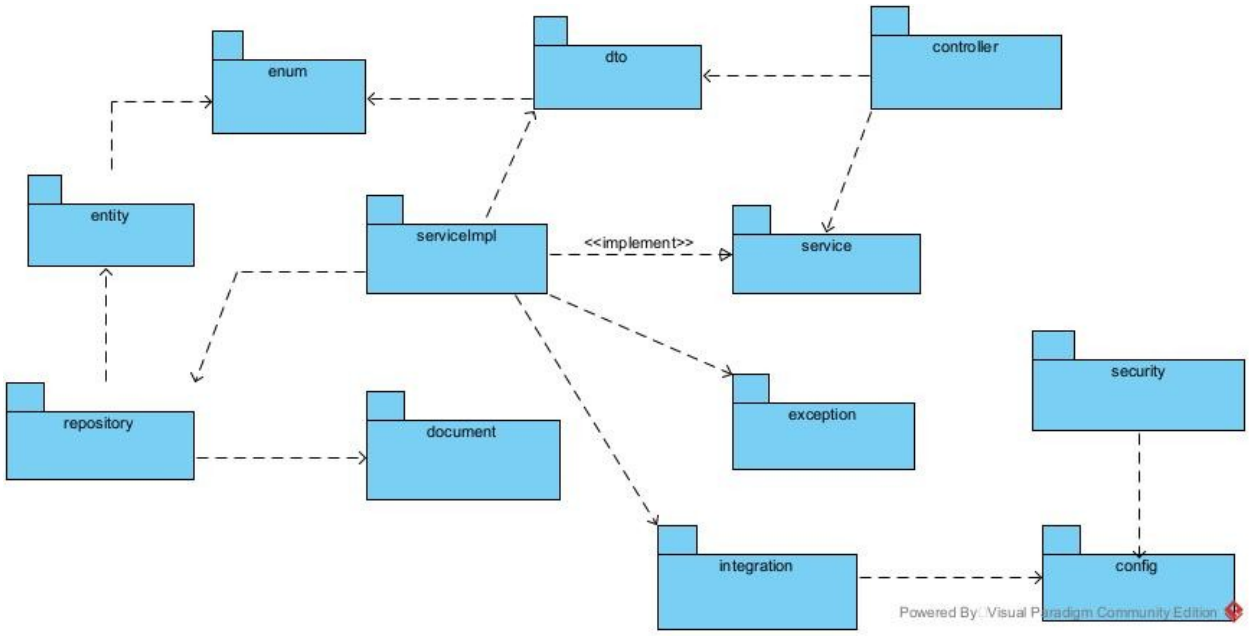


Figure 3-1: HMS Back-End Package Diagram

No	Package	Description
01	Config	Contains application configuration for CORS, OpenAPI, security, database access, payment, and mail services.
02	Security	Handles JWT authentication, password encoding, and role-based access control.
03	Controller	Contains REST controllers that receive HTTP requests and delegate processing to services.
04	DTO	Contains request and response objects used for data exchange between

		layers.
05	Entity	Contains JPA entity classes mapped to relational tables in Microsoft SQL Server.
06	Document	Contains MongoDB document models used for document-based data such as audit logs.
07	Enums	Defines shared constant values such as user roles, reservation status, room status, and payment status.
08	Exception	Contains custom exceptions and global exception handling components.
09	Repository	Contains Spring Data repositories for accessing SQL Server and MongoDB data.
10	Service	Defines service interfaces for business operations.
11	ServiceImpl	Implements service interfaces, business rules, and transaction processing.
12	Integration	Handles communication with external systems such as the Payment Gateway and Mail Notification Service.

3.2.3.1.2 Front-end package diagram

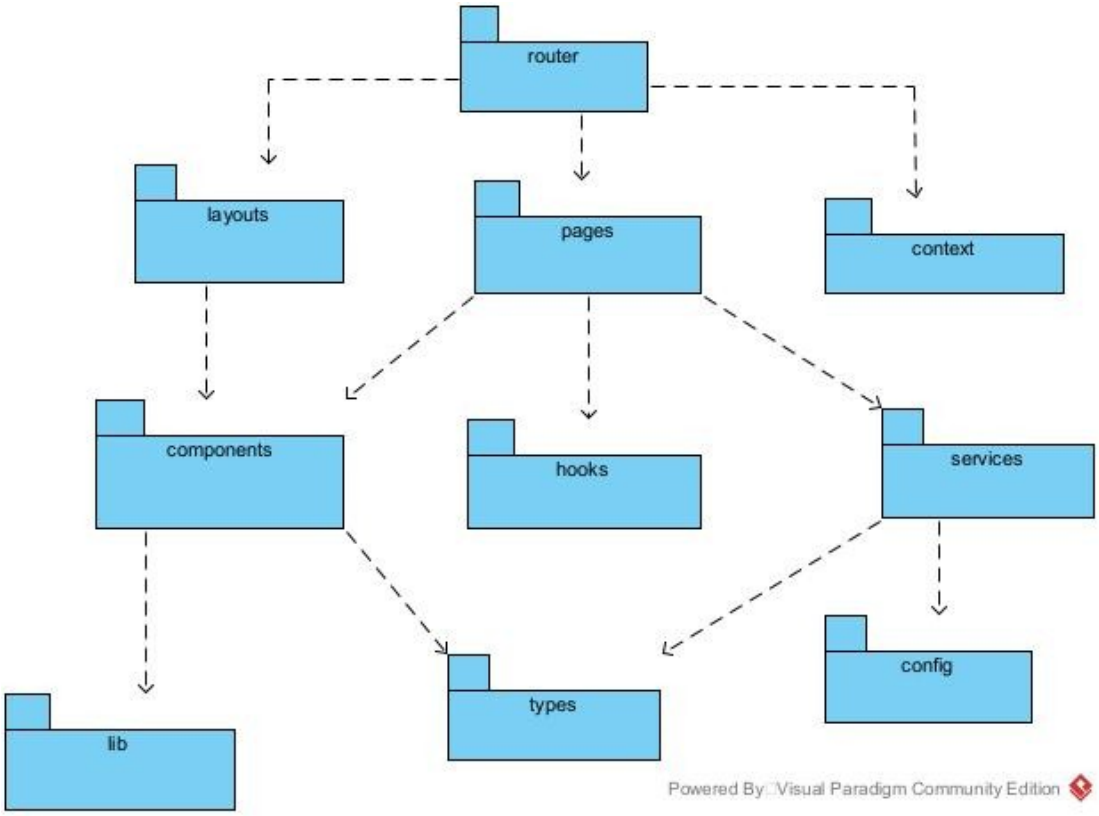


Figure 3-2: HMS Front-End Package Diagram

No	Package	Description
01	router	Manages application routing, navigation, protected routes, and role-based route access.
02	layouts	Defines shared page layouts such as headers, sidebars, navigation areas, and page containers.
03	pages	Contains page-level components corresponding to major HMS screens and application routes.
04	context	Manages shared application state such as authentication status, current user, and role information using React Context API.

05	components	Contains reusable UI components shared across multiple pages.
06	hooks	Contains reusable React Hooks for shared state handling, data loading, and frontend logic.
07	services	Handles communication with backend REST APIs and organizes API calls by functional module.
08	config	Contains frontend configuration such as API base URLs, environment settings, and shared constants.
09	lib	Contains reusable utility functions and helper logic used across frontend modules.
10	types	Defines shared data types and interfaces used by pages, components, and services.

3.2.4 Deployment View

Configuration Description

No	Component/Node	Description
01	< name>	<Description of the component>
02		

4 Detailed Component Design

4.1 Common Design

This section defines the standardized architectural mechanisms, shared components, and cross-cutting concerns applied across all five subsystems (50 use cases and 54 screens) of the Hotel Management

System (HMS). It covers centralized exception handling, audit logging, application message processing, frontend UI structures, shared utility components, and declarative transaction management.

4.1.1 Exception Handling

The HMS backend implements a centralized exception handling mechanism using Spring Boot's @RestControllerAdvice (GlobalExceptionHandler). This ensures that all runtime, validation, security, and business rule exceptions thrown from any Controller or Service are intercepted and transformed into a uniform JSON payload (ApiErrorResponse).

Response Payload Attributes:

- Timestamp (LocalDateTime): The exact UTC timestamp of the error.
- status (int): HTTP status code (e.g., 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict).
- ErrorCode (String): Standardized business rule code matching the SRS error catalogue (e.g., MSG01C, MSG04A, MSG08A, MSG09, MSG10).
- Message (String): Descriptive client-facing message.
- ValidationErrors (Map<String, String>): Key-value map detailing invalid field names and rejection reasons.

Core Custom Exceptions:

- ResourceNotFoundException: Thrown when an entity (Room, Reservation, Stay, Invoice) does not exist.
- BusinessException: Thrown when a business constraint is violated (e.g., uninspected room assignment, non-zero balance checkout).
- InvalidReservationStateException: Thrown during illegal lifecycle state transitions.
- UnauthorizedOperationException: Thrown when access is forbidden by RBAC.

4.1.2 Logging (Audit Logging)

- All monetary operations (deposit, payment, refund) and auditable administrative actions must be logged.
- The system persists these records as immutable AuditLog documents within a MongoDB (NoSQL) document database.
- These audit records follow a minimum 12-month retention policy, cannot be edited or deleted by operational staff, and are viewable only by the Administrator.

4.1.3 Messages Processing

- The system utilizes a standardized application message catalogue with predefined codes (MSG01A to MSG12B)
- Failed input validations keep the user on the same screen and return standardized inline messages.

- The user interface supports bilingual messaging: Vietnamese is the default for staff roles, and English is the default for guests/customers.

4.1.4 UI Structures

- The frontend is built as a Single Page Application (SPA) using ReactJS 18 with Functional Components, React Router DOM for client-side navigation, and React-Bootstrap for responsive UI layouts.
- The UI relies on common page layouts such as headers, navigation bars, sidebars, and page containers.
- To prevent memory bottlenecks, all list and search endpoints (e.g., room inventory, customer bookings, user accounts) must implement server-side pagination and filtering using Spring Data's Pageable and Page<T> interfaces.
- On forms, fields marked with * are mandatory and must be provided before actions like Confirm or Save can be executed.

4.1.5 Utility Classes/Components

- **Frontend**: Reusable React Hooks are provided for shared state handling and data loading, while common utility functions and helper logic are organized within the hooks and lib packages. Axios is utilized for sending asynchronous HTTP/HTTPS requests with JSON payloads.
- **Backend**: Data Transfer Objects (DTO) are used to encapsulate and transfer data between the presentation layer and the service layer. This ensures that database Entity objects are not exposed directly to the client via API payloads.

4.1.6 Transaction Management

- The backend enforces strict ACID Transaction Management using Spring Boot's @Transactional annotation.
- Multi-step state changes—such as Guest Check-in (which involves updating the Reservation status to Checked-In, creating a Guest Stay, initializing a Folio, and setting the Room status to Occupied), Room Transfers, Stay Extensions, and Check-outs—must be executed within atomic database transactions. If any failure occurs, all partial changes are rolled back.
- The design enforces "Configuration Isolation" for historical records: updates to room category base tariffs or hotel service unit prices must not alter completed reservations, historical service orders, or finalized invoices.

4.2 Account & Authentication Management

The Account & Authentication subsystem coordinates identity access management, user registration, role governance, credential recovery, and secure JWT-based stateless session management across all user personas (Customer, Front-Desk Staff, Housekeeping Staff, Administrator). This section details the detailed component design covering UC-01 through UC-10.

4.2.1 Screen Design

Figure 4-1 illustrates the user authentication screen. Table 4-1 provides the control specifications for the login interface.

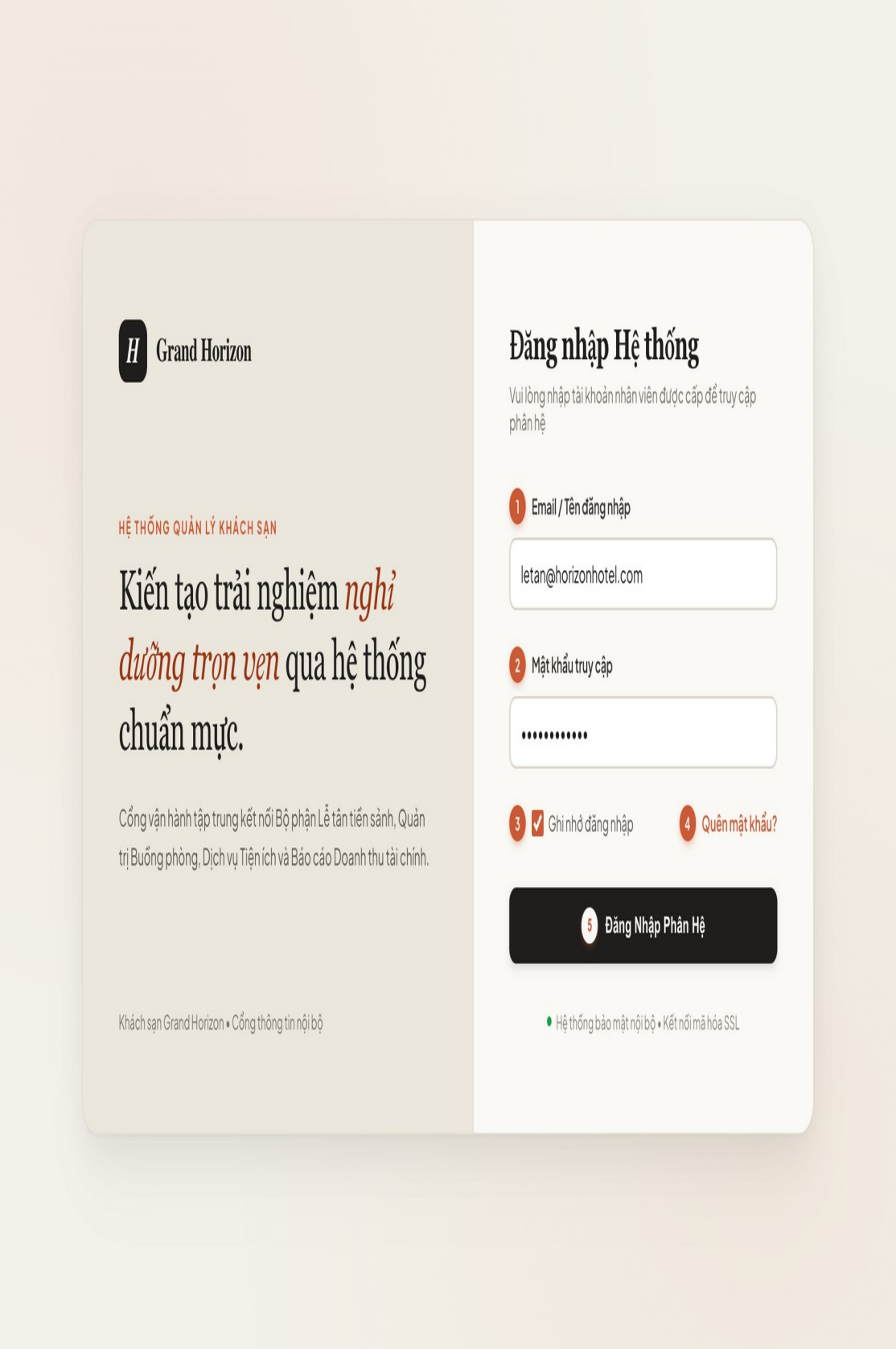

Grand Horizon
HỆ THỐNG QUẢN LÝ KHÁCH SẠN
Kiến tạo trải nghiệm nghỉ dưỡng trọn vẹn qua hệ thống chuẩn mực.
Cổng vận hành tập trung kết nối Bộ phận Lễ tân tiền sảnh, Quản trị Buồng phòng, Dịch vụ Tiện ích và Báo cáo Doanh thu tài chính.
Khách sạn Grand Horizon • Cổng thông tin nội bộ
Đăng nhập Hệ thống
Vui lòng nhập tài khoản nhân viên được cấp để truy cập phân hệ
1 Email / Tên đăng nhập
letan@horizonhotel.com
2 Mật khẩu truy cập
3 Ghi nhớ đăng nhập
4 Quên mật khẩu?
5 Đăng Nhập Phân Hệ
Hệ thống bảo mật nội bộ • Kết nối mã hóa SSL

Figure 4-1: Screen Design of User Login (UC-02)

Table 4-1: Screen Definition for User Login (UC-02)

No	Object/Control Name	Type	Required	Length	Description
(1)	**txtUsername**	TextField	Yes	4-100 chars	Input field for registered staff user email [DB Mapping: tbl_user.email]. Validates RFC 5322 syntax.
(2)	**txtPassword**	PasswordField	Yes	6-50 chars	Secure masked password input. Compared against BCrypt hash [DB Mapping: tbl_user.password_hash].
(3)	**chkRememberMe**	Checkbox	No	Default: false	Controls refresh token cookie expiration (7 days persistent vs session-only).
(4)	**lnkForgotPassword**	Hyperlink	No	Redirect	Navigates to secure staff password recovery workflow (UC-10).
(5)	**btnLogin**	Button	Yes	Trigger API	Initiates Step 1 in Sequence Diagram UC-02; dispatches HTTP POST /api/v1/auth/login [Auth Filter Validation].

4.2.2 Class/Component Design

4.2.2.1 Backend Class Diagram

The backend architecture strictly adheres to standard Java Spring Boot 3-Tier Enterprise patterns. The diagram uses flat layout and strictly applies standard UML relationship arrows: Realization (..|>) between ServiceImpl and Interface, Dependency (..>) from Controller to Service and Repository to Entity, and Association (-->) for Spring bean repository references and domain entity associations.

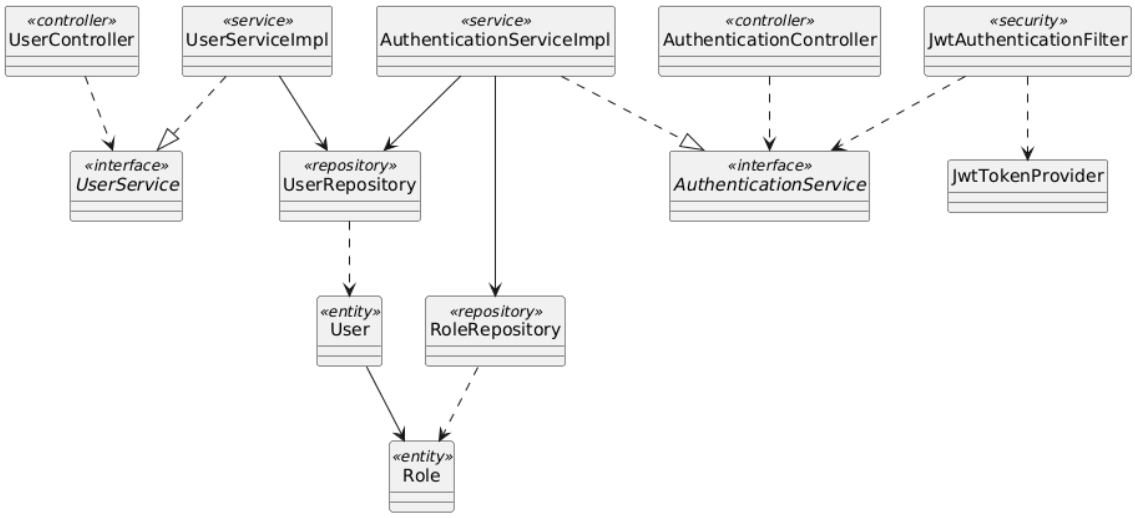


Figure 4-2: Backend Class/Component Structure Diagram for Account & Authentication

4.2.2.2 Classes/Component Description

4.2.2.2.1 AuthenticationController Class

Class	**AuthenticationController**			
Description	Exposes REST endpoints for user authentication, token issuance, and password recovery.			
Base Class	None			
Constructor	+ AuthenticationController(authService: AuthenticationService)			
Prototype	@RestController @RequestMapping("/api/v1/auth") public class AuthenticationController			
Source File	com/hotel/hms/controller/AuthenticationController.java			
Namespace	com.hotel.hms.controller			
Attributes	**Name**	**Type**	**Description**	
	authService	AuthenticationService	Business logic interface for credential authentication.	
Methods	**Name**	**Input**	**Output**	**Description**
	login	LoginRequestDTO request	ResponseEntity<AuthResponseDTO>	Authenticates credentials and returns JWT bearer token.
	resetPassword	ResetPasswordDTO request	ResponseEntity<Void>	Validates token and commits updated credential.

4.2.2.2.2 AuthenticationServiceImpl Class

Class	AuthenticationServiceImpl			
Description	Implements core authentication flows, BCrypt credential matching, and JWT token generation.			
Base Class	AuthenticationService			
Constructor	+ AuthenticationServiceImpl(userRepo: UserRepository, roleRepo: RoleRepository, tokenProvider: JwtTokenProvider)			
Prototype	@Service @Transactional public class AuthenticationServiceImpl implements AuthenticationService			
Source File	com/hotel/hms/service/impl/AuthenticationServiceImpl.java			
Namespace	com.hotel.hms.service.impl			
Attributes	**Name**	**Type**	**Description**	
	userRepository	UserRepository	Data access for user identity retrieval.	
	roleRepository	RoleRepository	Data access for user authorization roles.	
	jwtTokenProvider	JwtTokenProvider	Component generating HMAC-SHA signed JWT tokens.	
Methods	**Name**	**Input**	**Output**	**Description**
	authenticate	LoginRequestDTO dto	AuthResponseDTO	Verifies password hash and produces JWT accessToken with role claims.
	requestPasswordReset	String email	void	Generates reset token and invokes MailNotificationService.

4.2.2.2.3 UserRepository Class

Class	UserRepository
Description	Spring Data JPA repository providing database queries for User entity records.
Base Class	JpaRepository<User, Long>
Constructor	None (Spring Data JPA Interface)
Prototype	@Repository public interface UserRepository extends JpaRepository<User, Long>
Source File	com/hotel/hms/repository/UserRepository.java
Namespace	com.hotel.hms.repository

Attributes	**Name**	**Type**	**Description**	
	None	-	Inherits standard Spring Data JPA persistence context.	
Methods	**Name**	**Input**	**Output**	**Description**
	findByEmail	String email	Optional<User>	Retrieves active user account by unique email address.
	existsByEmail	String email	boolean	Checks uniqueness of email during account registration.

4.2.2.2.4 User Class

Class	**User**		
Description	JPA entity mapped to table 'User'. Represents staff and customer account credentials and operational status.		
Base Class	None		
Constructor	+ User()		
Prototype	@Entity @Table(name = "User") public class User		
Source File	com/hotel/hms/entity/User.java		
Namespace	com.hotel.hms.entity		
Attributes	**Name**	**Type**	**Description**
	userId	Long	Primary key identifier of user.
	email	String	Unique email address used for system login.
	passwordHash	String	BCrypt hashed secret key; never stored in plaintext.
	fullName	String	Full legal name of the user.
	role	Role	Many-to-One reference to Role entity.
	phone	String	Contact telephone number (mapped to User.phone in Section 5).
	status	String	Operational account status: ACTIVE, SUSPENDED,

			DELETED (mapped to User.status in Section 5).	
Methods	**Name**	**Input**	**Output**	**Description**
	getUserId	void	Long	Returns user primary key.
	getPasswordHash	void	String	Returns hashed credential.

4.2.3 Object Interactions (Sequence Diagram)

Figure 4-3 illustrates the runtime sequence flow for UC-02 User Login. Table 4-2 details each interaction step.

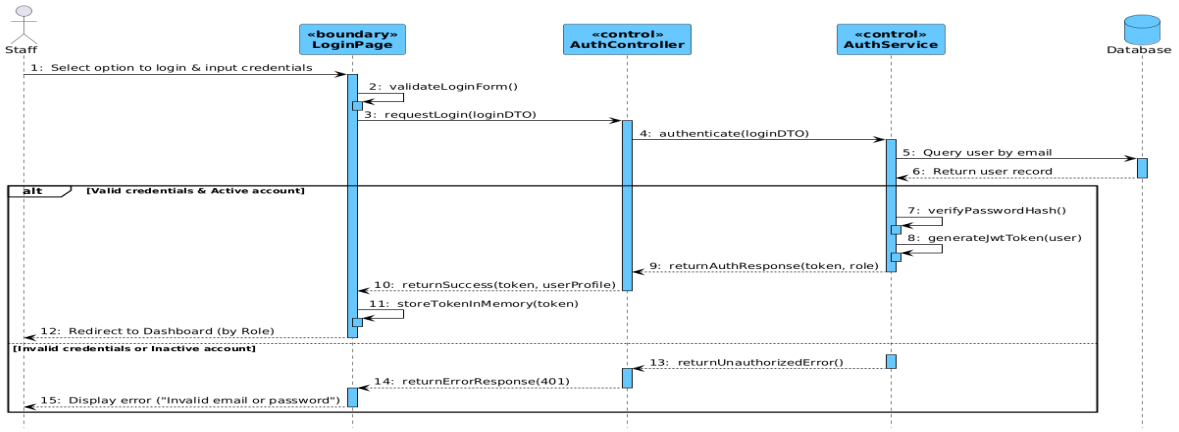


Figure 4-3: Sequence Diagram for UC-02 User Login Flow

Table 4-2: Interaction Steps for UC-02 User Login Flow

Step	Caller (From)	Callee (To)	Action / Method	Description
1	Staff	LoginPage	Select option to login & input credentials	Staff accesses login screen and enters email and password.
2	LoginPage	LoginPage	validateLoginForm()	Validates client-side input for non-empty fields and valid email format.
3	LoginPage	AuthController	requestLogin(loginDTO)	Dispatches HTTP POST request with authentication credentials payload.
4	AuthController	AuthService	authenticate(loginDTO)	Delegates credential authentication logic to security service layer.

5	AuthService	Database	Query user by email	Retrieves active user account entity by unique email address.
6	Database	AuthService	Return user record	Returns user record with encrypted password hash and assigned role.
7 [Success]	AuthService	AuthService	verifyPasswordHash()	Verifies submitted password against stored BCrypt hash.
8 [Success]	AuthService	AuthService	generateJwtToken(user)	Signs and generates JWT token containing user identity and role claims.
9 [Success]	AuthService	AuthController	returnAuthResponse(token, role)	Returns authentication response DTO with generated JWT token.
10 [Success]	AuthController	LoginPage	returnSuccess(token, userProfile)	Responds with HTTP 200 OK and authenticated user profile payload.
11 [Success]	LoginPage	LoginPage	storeTokenInMemory(token)	Saves JWT token in client state/memory for subsequent API calls.
12 [Success]	LoginPage	Staff	Redirect to Dashboard (by Role)	Redirects user to the designated portal based on their assigned role.
13 [Failure]	AuthService	AuthController	returnUnauthorizedError()	Throws BadCredentialsException when email/password is invalid or inactive.
14 [Failure]	AuthController	LoginPage	returnErrorResponse(401)	Responds with HTTP 401 Unauthorized status and error code.
15 [Failure]	LoginPage	Staff	Display error ("Invalid email or password")	Displays localized error toast notification on login form.

4.3 Room & Facility Management

The Room & Facility Management subsystem governs physical accommodation categories, room units, floor indexing, real-time cleanliness states, and maintenance lockouts (UC-11 through UC-20). This section details the complete technical design focusing on UC-19 (Update Housekeeping Status) and UC-20 (Set Room Maintenance).

4.3.1 Screen Design

Figures 4-4 and 4-5 show the screen designs for the Housekeeping Board and Set Room Maintenance Form. Tables 4-3 and 4-4 define their controls.

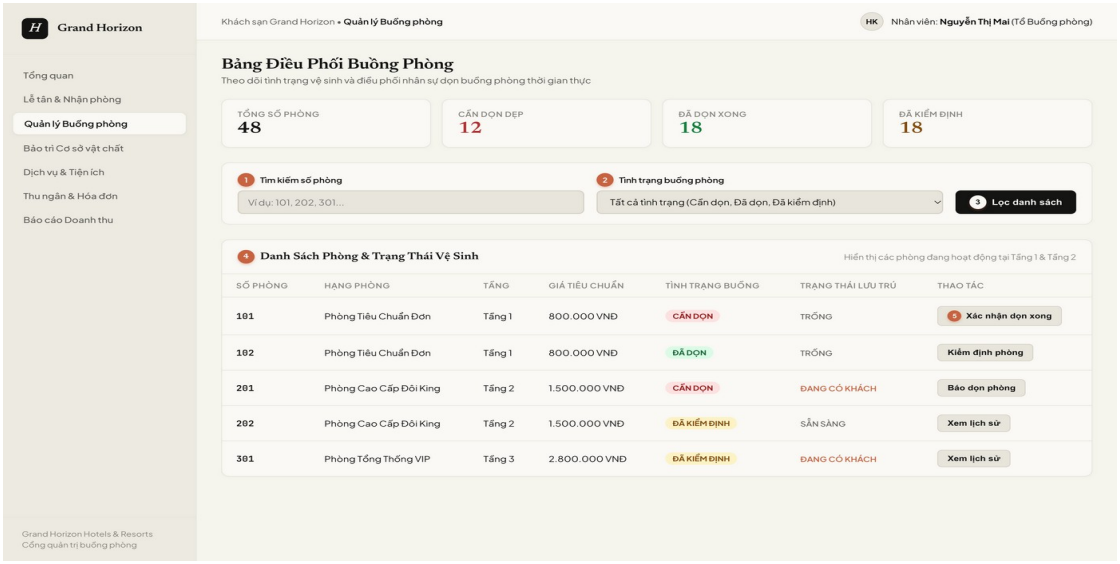


Figure 4-4: Screen Design of Housekeeping Board (UC-19)

Table 4-3: Screen Definition for Housekeeping Board (UC-19)

No	Object/Control Name	Type	Required	Length	Description
(1)	**txtSearchRoomNo**	TextBox	No	50 chars	Keyword search filtering physical rooms by door number [DB Mapping: tbl_room.room_number].
(2)	**cboHousekeepingStatus**	Dropdown	No	N/A	Filter dropdown for sanitation criteria [DB Mapping: tbl_room.status in (DIRTY, CLEAN, INSPECTED)].
(3)	**btnFilter**	Button	Yes	Trigger API	Applies search criteria and executes GET /api/v1/housekeeping/rooms to re-render grid.
(4)	**gridRoomList**	Grid / Table	Yes	N/A	Dynamic room sanitation dataset [DB Mapping: tbl_room INNER JOIN tbl_room_type ON room_type_id].

No	Object/Control Name	Type	Required	Length	Description
(5)	**btnConfirmClean**	Button	Yes	Trigger API	Row action initiating Step 1 in Sequence Diagram UC-19; transitions room state [DB Action: UPDATE tbl_room.status = 'CLEAN'].

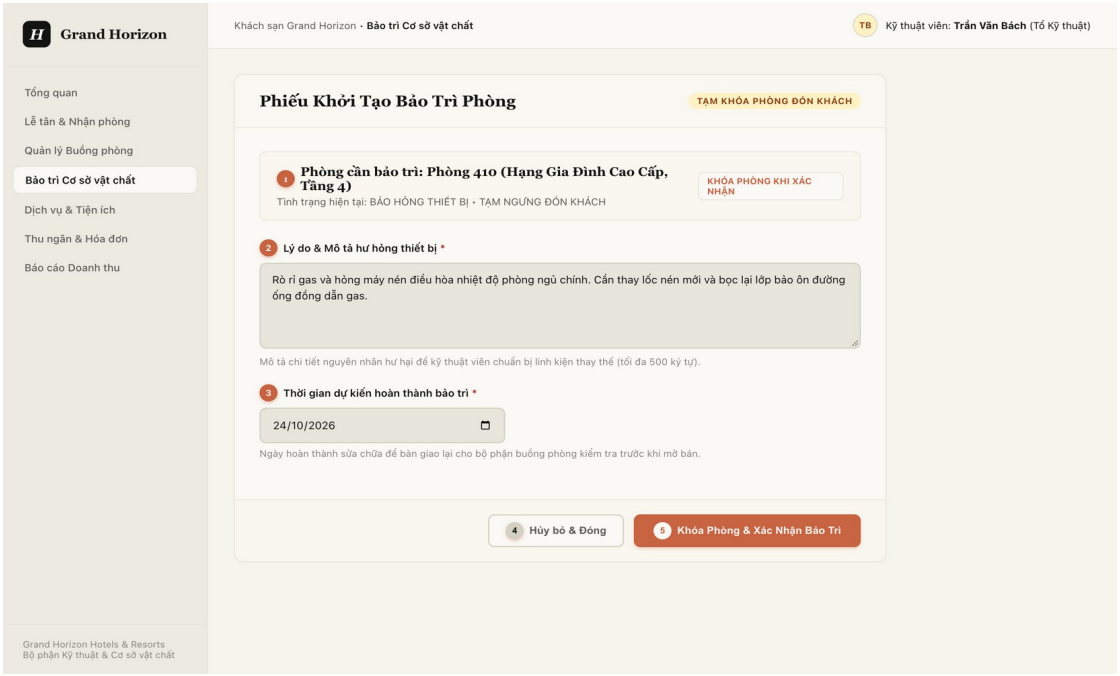


Figure 4-5: Screen Design of Set Room Maintenance Form (UC-20)

Table 4-4: Screen Definition for Set Room Maintenance Form (UC-20)

No	Object/Control Name	Type	Required	Length	Description
(1)	**lblRoomSelected**	Label / Display	Yes	N/A	Displays target physical room number and category [DB Mapping: tbl_room.room_number, tbl_room_type.type_name].
(2)	**txtMaintenanceReason**	TextArea	Yes	500 chars	Mandatory technical explanation of defect [DB Mapping: tbl_room_maintenance.note] (*). Enforces Rule BR-16.
(3)	**dtMaintainedAt**	DatePicker	Yes	YYYY-MM-DD	Target completion date [DB Mapping: tbl_room_maintenance.maintained_at] (*).

No	Object/Control Name	Type	Required	Length	Description
(4)	**btnCancel**	Button	No	Redirect	Cancels maintenance order creation and returns to room overview.
(5)	**btnConfirmMaintenance**	Button	Yes	Trigger API	Dispatches POST /api/v1/rooms/maintenance [DB Action: INSERT tbl_room_maintenance, UPDATE tbl_room.status = 'UNDER_MAINTENANCE'].

4.3.2 Class/Component Design

4.3.2.1 Backend Class Diagram

Figure 4-6 presents the backend class diagram for Room & Facility Management using flat layout and rigorous UML relationships: Realization (..|>) for services, Dependency (..>) for controller and repository interactions, and Association (-->) for repository references and entity associations.

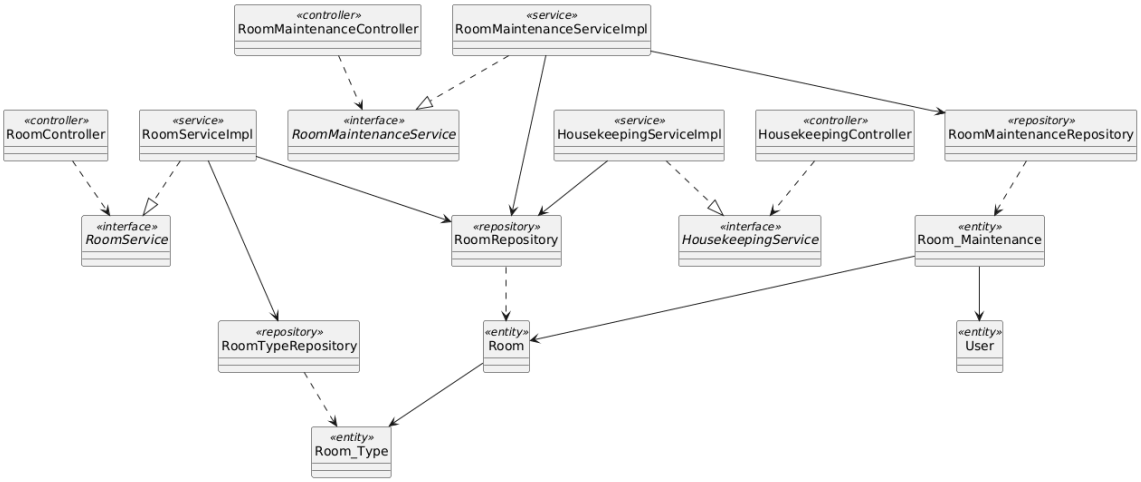


Figure 4-6: Backend Class/Component Structure Diagram for Room & Facility Management

4.3.2.2 Classes/Component Description

4.3.2.2.1 HousekeepingController Class

Class	HousekeepingController
Description	REST controller exposing endpoints for housekeeping board retrieval and room hygiene updates.
Base Class	None
Constructor	+ HousekeepingController(housekeepingService: HousekeepingService)
Prototype	@RestController @RequestMapping("/api/v1/housekeeping") public class HousekeepingController

Source File	com/hotel/hms/controller/HousekeepingController.java			
Namespace	com.hotel.hms.controller			
Attributes	**Name**	**Type**	**Description**	
	housekeepingService	HousekeepingService	Service handling cleanliness transitions.	
Methods	**Name**	**Input**	**Output**	**Description**
	updateCleanliness	Long roomId, CleanlinessDTO dto	ResponseEntity<RoomDTO>	Updates room status to CLEAN.
	getBoard	Integer floor, String status	ResponseEntity<List<RoomDTO>>	Retrieves rooms filtered by floor and status.

4.3.2.2.2 RoomMaintenanceServiceImpl Class

Class	RoomMaintenanceServiceImpl			
Description	Service executing room maintenance scheduling, guest occupancy conflict verification, and inventory locking.			
Base Class	RoomMaintenanceService			
Constructor	+ RoomMaintenanceServiceImpl(maintenanceRepo: RoomMaintenanceRepository, roomRepo: RoomRepository)			
Prototype	@Service @Transactional public class RoomMaintenanceServiceImpl implements RoomMaintenanceService			
Source File	com/hotel/hms/service/impl/RoomMaintenanceServiceImpl.java			
Namespace	com.hotel.hms.service.impl			
Attributes	**Name**	**Type**	**Description**	
	roomMaintenanceRepository	RoomMaintenanceRepository	Persistence repository for maintenance records.	
	roomRepository	RoomRepository	Persistence repository for room operational status.	
Methods	**Name**	**Input**	**Output**	**Description**
	scheduleMaintenance	Long roomId, MaintenanceDTO dto	MaintenanceResponseDTO	Checks occupancy, creates RoomMaintenance record, and sets room status to UNDER_MAINTENANCE.

	checkOccupancyStatus	Room room	void	Throws RoomOccupiedException if room currently has active guest stay.

4.3.2.2.3 RoomRepository Class

Class	**RoomRepository**			
Description	Spring Data JPA repository managing physical room records and operational states.			
Base Class	JpaRepository<Room, Long>			
Constructor	None (Spring Data JPA Interface)			
Prototype	@Repository public interface RoomRepository extends JpaRepository<Room, Long>			
Source File	com/hotel/hms/repository/RoomRepository.java			
Namespace	com.hotel.hms.repository			
Attributes	**Name**	**Type**	**Description**	
	None	-	Inherits Spring Data JPA persistence context.	
Methods	**Name**	**Input**	**Output**	**Description**
	findByRoomNumber	String roomNumber	Optional<Room>	Finds room record by unique door number.
	updateRoomStatus	Long roomId, String status	int	Modifies room operational status in SQL Server.

4.3.2.2.4 Room Class

Class	**Room**
Description	JPA entity mapped to table 'Room'. Stores physical room number, room type reference, and status.
Base Class	None
Constructor	+ Room()
Prototype	@Entity @Table(name = "Room") public class Room
Source File	com/hotel/hms/entity/Room.java
Namespace	com.hotel.hms.entity

Attributes	Name	Type	Description	
	roomId	Long	Primary key identifier of room.	
	roomNumber	String	Unique door number (e.g., '301').	
	status	String	Current state: VACANT, OCCUPIED, DIRTY, CLEAN, UNDER_MAINTENANCE.	
	roomType	Room_Type	Many-to-One reference to Room_Type entity.	
Methods	**Name**	**Input**	**Output**	**Description**
	isAvailable	void	boolean	Returns true if room is VACANT and CLEAN.
	setStatus	String status	void	Sets new operational status.

4.3.3 Object Interactions (Sequence Diagram)

Figures 4-7 and 4-8 depict runtime sequence interactions for UC-19 and UC-20. Tables 4-5 and 4-6 list each interaction step.

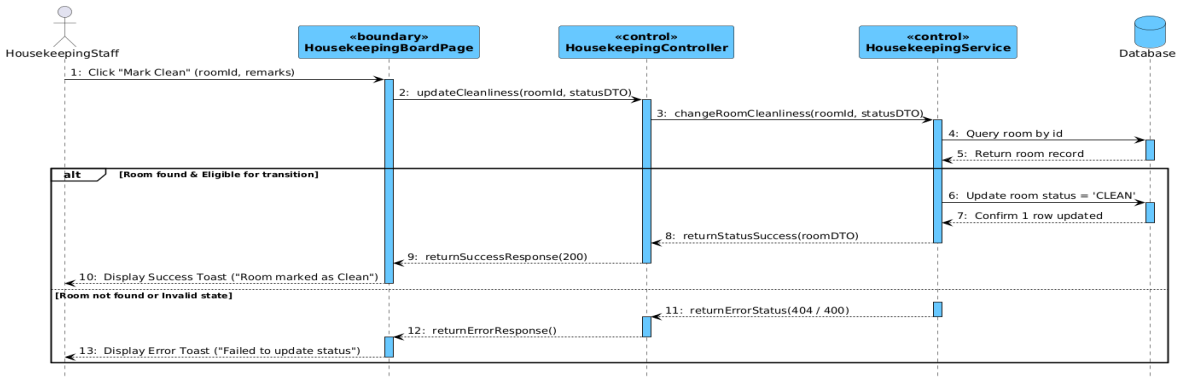


Figure 4-7: Sequence Diagram for UC-19 Update Housekeeping Status

Table 4-5: Interaction Steps for UC-19 Update Housekeeping Status

Step	Caller (From)	Callee (To)	Action / Method	Description
1	HousekeepingStaff	HousekeepingBoardPage	Click "Mark Clean" (roomId, remarks)	Staff selects room on housekeeping board and submits cleanliness update.
2	HousekeepingBoardPage	HousekeepingController	updateCleanliness(roomId, statusDTO)	Dispatches HTTP PUT request with target status and remarks.

3	HousekeepingController	HousekeepingService	changeRoomCleanliness(roomId, statusDTO)	Delegates room cleanliness state transition to housekeeping service.
4	HousekeepingService	Database	Query room by id	Retrieves physical room entity and validates current status.
5	Database	HousekeepingService	Return room record	Returns room entity with current status and room category.
6 [Success]	HousekeepingService	Database	Update room status = 'CLEAN'	Updates room status to CLEAN and logs cleaning timestamp.
7 [Success]	Database	HousekeepingService	Confirm 1 row updated	Database confirms successful record update.
8 [Success]	HousekeepingService	HousekeepingController	returnStatusSuccess(roomDTO)	Returns updated room state DTO.
9 [Success]	HousekeepingController	HousekeepingBoardPage	returnSuccessResponse(200)	Responds with HTTP 200 OK status.
10 [Success]	HousekeepingBoardPage	HousekeepingStaff	Display Success Toast ("Room marked as Clean")	Displays success toast notification and updates room badge to Clean.
11 [Failure]	HousekeepingService	HousekeepingController	returnErrorStatus(404 / 400)	Returns error when room is not found or status transition is invalid.
12 [Failure]	HousekeepingController	HousekeepingBoardPage	returnErrorResponse()	Responds with HTTP 400/404 error response.
13 [Failure]	HousekeepingBoardPage	HousekeepingStaff	Display Error Toast ("Failed to update status")	Renders error toast notification on housekeeping board.

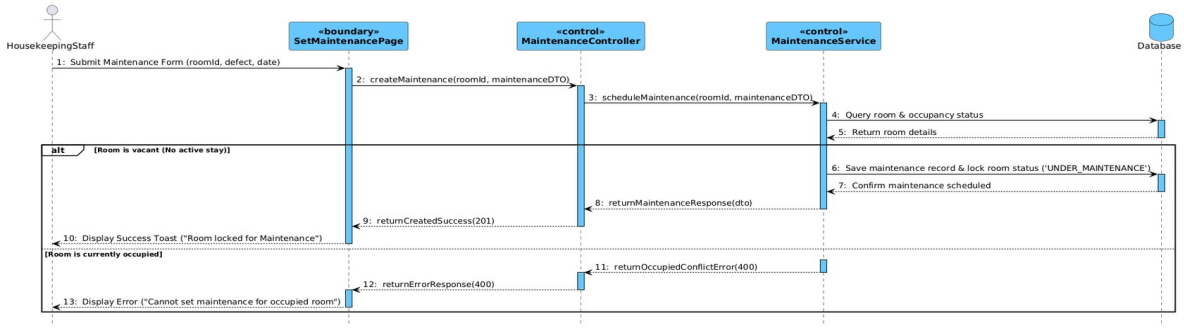


Figure 4-8: Sequence Diagram for UC-20 Set Room Maintenance

Table 4-6: Interaction Steps for UC-20 Set Room Maintenance

Step	Caller (From)	Callee (To)	Action / Method	Description
1	HousekeepingStaff	SetMaintenancePage	Submit Maintenance Form (roomId, defect, date)	Staff submits maintenance form with defect reason and expected completion.
2	SetMaintenancePage	MaintenanceController	createMaintenance(roomId, maintenanceDTO)	Dispatches HTTP POST request to schedule maintenance ticket.
3	MaintenanceController	MaintenanceService	scheduleMaintenance(roomId, maintenanceDTO)	Delegates maintenance scheduling logic to maintenance service layer.
4	MaintenanceService	Database	Query room & occupancy status	Checks room occupancy to ensure no active guest stay is present.
5	Database	MaintenanceService	Return room details	Returns room record with current status and stay assignments.
6 [Success]	MaintenanceService	Database	Save maintenance record & lock room status ('UNDER_MAINTENANCE')	Atomically inserts maintenance order and updates room status to UNDER_MAINTENANCE.
7 [Success]	Database	MaintenanceService	Confirm maintenance scheduled	Database confirms maintenance transaction committed.
8 [Success]	MaintenanceService	MaintenanceController	returnMaintenanceRes	Returns maintenance

			ponse(dto)	ticket response DTO.
9 [Success]	MaintenanceController	SetMaintenancePage	returnCreatedSuccess(201)	Responds with HTTP 201 Created status.
10 [Success]	SetMaintenancePage	HousekeepingStaff	Display Success Toast ("Room locked for Maintenance")	Closes modal and updates room card status to Under Maintenance.
11 [Failure]	MaintenanceService	MaintenanceController	returnOccupiedConflictError(400)	Returns conflict error when room is currently occupied by guests.
12 [Failure]	MaintenanceController	SetMaintenancePage	returnErrorResponse(400)	Responds with HTTP 400 Bad Request error.
13 [Failure]	SetMaintenancePage	HousekeepingStaff	Display Error ("Cannot set maintenance for occupied room")	Displays validation error prohibiting maintenance lock on occupied room.

4.4 Reservation & Front-Desk Operations

The Reservation & Front-Desk subsystem manages guest accommodation discovery, online reservation checkout, front-desk arrival check-in, physical room key allocation, and active guest stay tracking (UC-21 through UC-30). This section details UC-21/23 (Room Reservation) and UC-27 (Guest Check-in).

4.4.1 Screen Design

Figures 4-9 and 4-10 illustrate the user interfaces for Booking Checkout and Guest Check-in. Tables 4-7 and 4-8 define their controls.

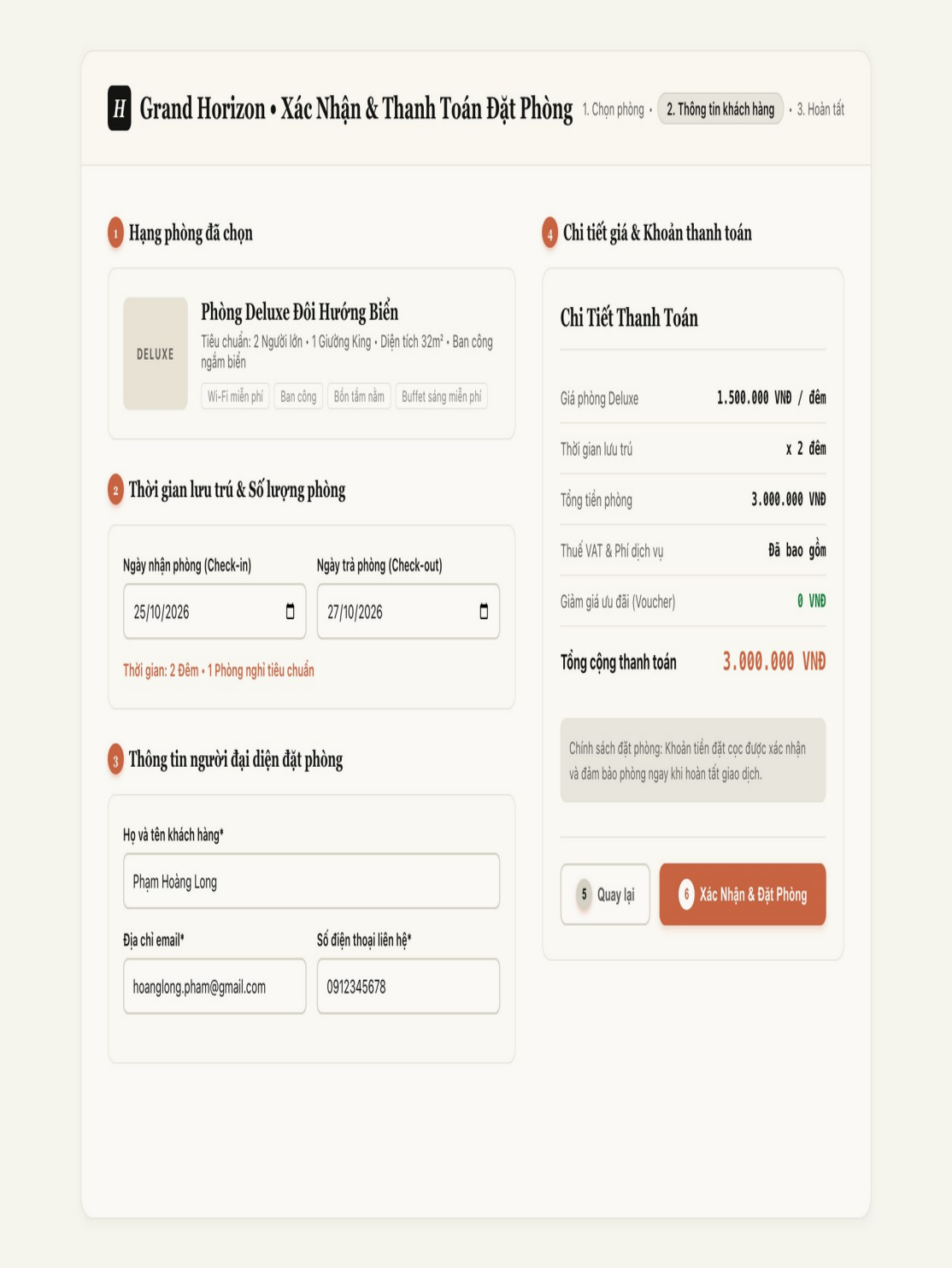


Figure 4-9: Screen Design of Booking Checkout (UC-23)

Table 4-7: Screen Definition for Booking Checkout (UC-23)

No	Object/Control Name	Type	Required	Length	Description
(1)	**pnlRoomOverview**	Display / Panel	Yes	N/A	Displays reserved room category, capacity, and base price [DB Mapping: tbl_room_type.type_name, capacity, base_price].
(2)	**pnlBookingDates**	Input / Panel	Yes	N/A	Selects scheduled stay interval and room count [DB Mapping: tbl_reservation.check_in_date, check_out_date, tbl_reservation_detail.quantity].
(3)	**pnlGuestContact**	Input / Panel	Yes	N/A	Primary guest contact information fields [DB Mapping: tbl_reservation.contact_name, contact_email, contact_phone].
(4)	**tblPriceDetails**	Display / Table	Yes	Currency	Itemized tariff calculation [DB Mapping: tbl_reservation_detail.agreed_rate * duration; tbl_payment.amount deposit].
(5)	**btnBack**	Button	No	Redirect	Navigates back to room search and category selection catalog.
(6)	**btnConfirmBooking**	Button	Yes	Trigger API	Initiates Step 1 in Sequence Diagram UC-23 [DB Action: INSERT tbl_reservation (status='CONFIRMED'), INSERT tbl_reservation_detail, INSERT tbl_payment].

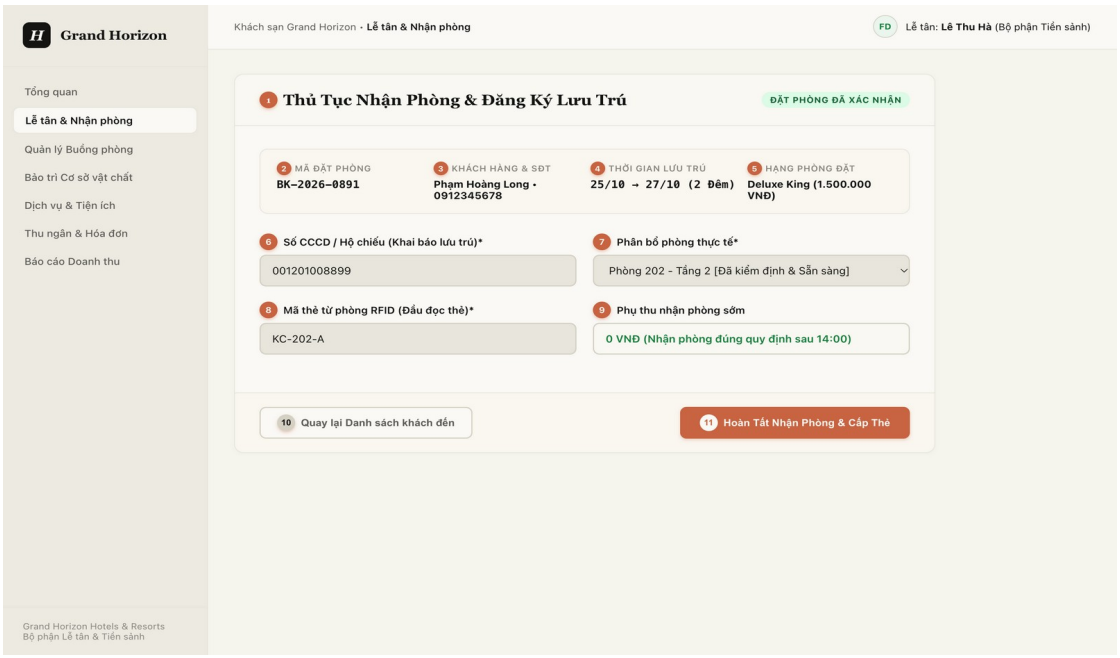


Figure 4-10: Screen Design of Guest Check-in Form (UC-27)

Table 4-8: Screen Definition for Guest Check-in Form (UC-27)

No	Object/Control Name	Type	Required	Length	Description
(1)	**lblCheckinTitle**	Label / Header	Yes	N/A	Header identifying the active check-in registration workflow.
(2)	**txtBookingCode**	Display / Text	Yes	20 chars	Displays verified reservation code [DB Mapping: tbl_reservation.reservation_code] .
(3)	**txtGuestProfile**	Display / Text	Yes	100 chars	Displays registered guest full name and phone number [DB Mapping: tbl_customer.full_name, phone_number].
(4)	**txtStayInterval**	Display / Date	Yes	N/A	Displays check-in / check-out dates and night duration [DB Mapping: tbl_guest_stay.actual_check_in, expected_check_out].
(5)	**txtRoomCategory**	Display / Text	Yes	50 chars	Displays reserved room category name and standard rate [DB Mapping: tbl_room_type.type_name].

No	Object/Control Name	Type	Required	Length	Description
(6)	**txtIdentityCard**	TextBox	Yes	9-20 chars	Mandatory Citizen ID / Passport number for legal lodging registry [DB Mapping: tbl_customer.identity_card_number] (*).
(7)	**cboAssignRoom**	Dropdown	Yes	N/A	Selects clean, inspected room for assignment [DB Mapping: tbl_room.room_id; DB Action: INSERT tbl_stay_room_assignment] (*).
(8)	**txtKeyCardNumber**	TextBox	Yes	20 chars	Physical RFID card number encoded for door access [DB Action: RFID Card Encoder Sync] (*).
(9)	**txtEarlySurcharge**	Display / Currency	No	N/A	Calculated early check-in surcharge if arriving before 14:00 [DB Mapping: tbl_additional_charge.amount].
(10)	**btnBackToArrivals**	Button	No	Redirect	Returns to front-desk pending arrivals queue without saving.
(11)	**btnCompleteCheckin**	Button	Yes	Trigger API	Initiates Step 1 in Sequence Diagram UC-27 [DB Action: INSERT tbl_guest_stay, INSERT tbl_stay_room_assignment, UPDATE tbl_room.status='OCCUPIED', INSERT tbl_folio].

4.4.2 Class/Component Design

4.4.2.1 Backend Class Diagram

Figure 4-11 illustrates the backend class structure for Reservation and Front-Desk operations. Crucially, it utilizes UML Composition (*--) for lifecycle-dependent parent-child entities: Reservation *-- Reservation_Detail and Guest_Stay *-- Stay_Room_Assignment, alongside Realization (..|>) for services and Association (-->) for repository references and domain entity associations.

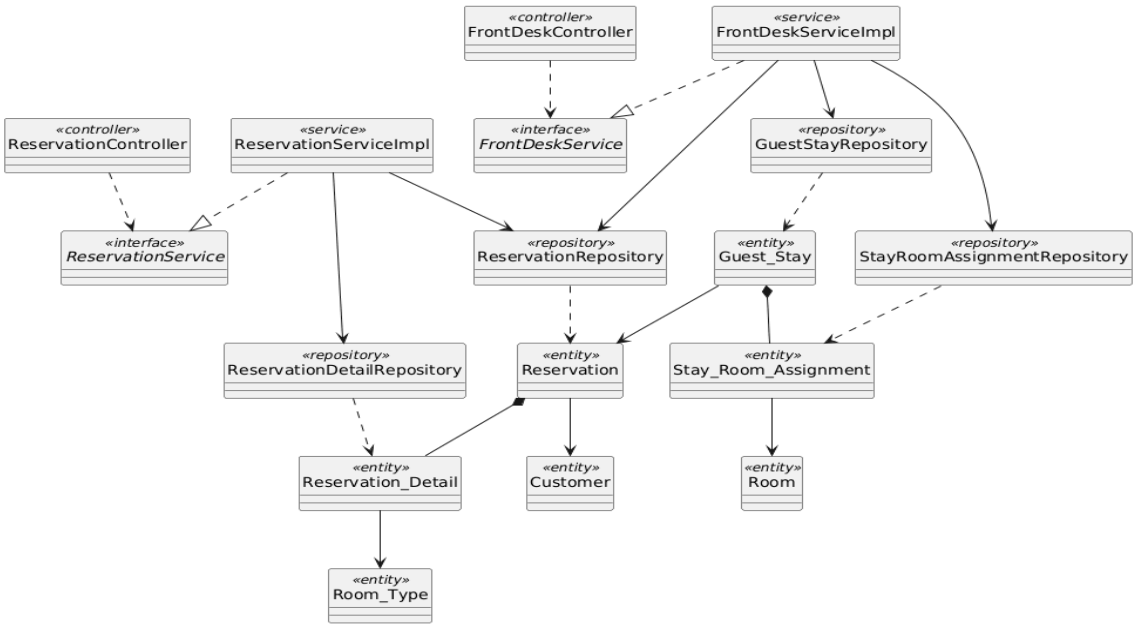


Figure 4-11: Backend Class/Component Structure Diagram for Reservation & Front-Desk

4.4.2.2 Classes/Component Description

4.4.2.2.1 ReservationController Class

Class	**ReservationController**			
Description	REST controller exposing endpoints for booking creation, cancellation, and guest booking inquiries.			
Base Class	None			
Constructor	+ ReservationController(reservationService: ReservationService)			
Prototype	@RestController @RequestMapping("/api/v1/reservations") public class ReservationController			
Source File	com/hotel/hms/controller/ReservationController.java			
Namespace	com.hotel.hms.controller			
Attributes	**Name**	**Type**	**Description**	
	reservationService	ReservationService	Business logic interface for room reservations.	
Methods	**Name**	**Input**	**Output**	**Description**
	createReservation	ReservationRequestDTO dto	ResponseEntity<ReservationResponseDTO>	Validates quota and creates room reservation.
	getReservation	Long id	ResponseEntity<ReservationResponseDTO>	Retrieves reservation itinerary details.

4.4.2.2.2 FrontDeskServiceImpl Class

Class	FrontDeskServiceImpl			
Description	Service executing front-desk arrival check-in, physical room assignment, and Guest_Stay creation.			
Base Class	FrontDeskService			
Constructor	+ FrontDeskServiceImpl(resRepo: ReservationRepository, stayRepo: GuestStayRepository, roomRepo: RoomRepository, assignRepo: StayRoomAssignmentRepository)			
Prototype	@Service @Transactional public class FrontDeskServiceImpl implements FrontDeskService			
Source File	com/hotel/hms/service/impl/FrontDeskServiceImpl.java			
Namespace	com.hotel.hms.service.impl			
Attributes	**Name**	**Type**	**Description**	
	reservationRepository	ReservationRepository	Access to reservation records.	
	guestStayRepository	GuestStayRepository	Persistence for active guest stay records.	
	roomRepository	RoomRepository	Persistence for updating room operational status.	
	stayRoomAssignmentRepository	StayRoomAssignmentRepository	Persistence for room door allocations.	
Methods	**Name**	**Input**	**Output**	**Description**
	performCheckIn	CheckInRequestDTO dto	CheckInResponseDTO	Allocates room, creates GuestStay, updates room status to OCCUPIED, opens folio.
	transferRoom	Long stayId, Long newRoomId	void	Transfers guest stay to new room and marks old room DIRTY.

4.4.2.2.3 ReservationRepository Class

Class	ReservationRepository
Description	Spring Data JPA repository managing Reservation entity records and booking status queries.
Base Class	JpaRepository<Reservation, Long>
Constructor	None (Spring Data JPA Interface)
Prototype	@Repository public interface ReservationRepository extends JpaRepository<Reservation, Long>

Source File	com/hotel/hms/repository/ReservationRepository.java			
Namespace	com.hotel.hms.repository			
Attributes	**Name**	**Type**	**Description**	
	None	-	Inherits Spring Data JPA persistence context.	
Methods	**Name**	**Input**	**Output**	**Description**
	findByReservationId	Long id	Optional<Reservation>	Queries reservation by ID.
	findActiveBookingsByDate	LocalDate date	List<Reservation>	Queries active bookings for arrival dispatch.

4.4.2.2.4 Guest_Stay Class

Class	Guest_Stay			
Description	JPA entity mapped to table 'Guest_Stay'. Records actual check-in and check-out timestamps of an active stay.			
Base Class	None			
Constructor	+ Guest_Stay()			
Prototype	@Entity @Table(name = "Guest_Stay") public class Guest_Stay			
Source File	com/hotel/hms/entity/Guest_Stay.java			
Namespace	com.hotel.hms.entity			
Attributes	**Name**	**Type**	**Description**	
	stayId	Long	Primary key identifier of guest stay.	
	actualCheckIn	LocalDateTime	Recorded check-in arrival timestamp.	
	actualCheckOut	LocalDateTime	Recorded departure timestamp.	
	status	String	Stay status: CHECKED_IN, CHECKED_OUT.	
	reservation	Reservation	Many-to-One reference to Reservation entity.	
Methods	**Name**	**Input**	**Output**	**Description**
	getStayId	void	Long	Returns stay primary key.

	setStatus	String status	void	Sets stay status.

4.4.3 Object Interactions (Sequence Diagram)

Figures 4-12 and 4-13 depict the runtime interactions for Room Reservation and Guest Check-in. Tables 4-9 and 4-10 list each step.

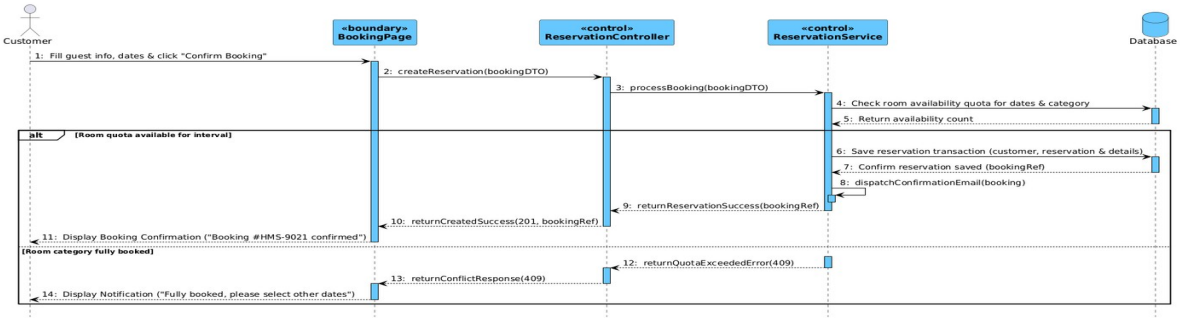


Figure 4-12: Sequence Diagram for UC-23 Room Reservation

Table 4-9: Interaction Steps for UC-23 Room Reservation

Step	Caller (From)	Callee (To)	Action / Method	Description
1	Customer	BookingPage	Fill guest info, dates & click "Confirm Booking"	Guest fills contact details, check-in/out dates, and chooses room category.
2	BookingPage	ReservationController	createReservation(bookingDTO)	Dispatches HTTP POST request with reservation payload.
3	ReservationController	ReservationService	processBooking(bookingDTO)	Delegates booking business rules and allocation to reservation service.
4	ReservationService	Database	Check room availability quota for dates & category	Queries inventory count to verify capacity for requested interval.
5	Database	ReservationService	Return availability count	Returns remaining vacant room count for category.
6 [Success]	ReservationService	Database	Save reservation transaction (customer, reservation & details)	Atomically creates customer profile, reservation header, and room detail rows.
7 [Success]	Database	ReservationService	Confirm reservation	Database confirms booking transaction

			saved (bookingRef)	and returns reference code.
8 [Success]	ReservationService	ReservationService	dispatchConfirmationEmail(booking)	Asynchronously triggers booking confirmation email to customer.
9 [Success]	ReservationService	ReservationController	returnReservationSuccess(bookingRef)	Returns reservation confirmation response DTO.
10 [Success]	ReservationController	BookingPage	returnCreatedSuccess(201, bookingRef)	Responds with HTTP 201 Created and booking reference.
11 [Success]	BookingPage	Customer	Display Booking Confirmation ("Booking #HMS-9021 confirmed")	Renders booking confirmation screen with code and instructions.
12 [Failure]	ReservationService	ReservationController	returnQuotaExceededError(409)	Returns inventory conflict error when requested room type is fully booked.
13 [Failure]	ReservationController	BookingPage	returnConflictResponse(409)	Responds with HTTP 409 Conflict status.
14 [Failure]	BookingPage	Customer	Display Notification ("Fully booked, please select other dates")	Displays alert requesting customer to choose alternative dates or categories.

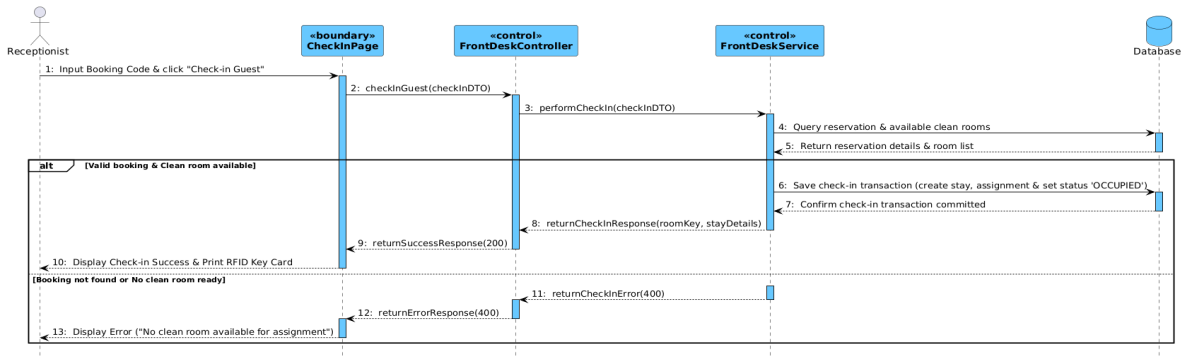


Figure 4-13: Sequence Diagram for UC-27 Guest Check-in

Table 4-10: Interaction Steps for UC-27 Guest Check-in

Step	Caller (From)	Callee (To)	Action / Method	Description

1	Receptionist	CheckInPage	Input Booking Code & click "Check-in Guest"	Front-desk receptionist enters booking code and initiates check-in.
2	CheckInPage	FrontDeskController	checkInGuest(checkInDTO)	Dispatches HTTP POST request with check-in payload.
3	FrontDeskController	FrontDeskService	performCheckIn(checkInDTO)	Transfers check-in workflow to front-desk service layer.
4	FrontDeskService	Database	Query reservation & available clean rooms	Verifies booking status and searches for available CLEAN rooms of category.
5	Database	FrontDeskService	Return reservation details & room list	Returns verified reservation details and eligible room candidates.
6 [Success]	FrontDeskService	Database	Save check-in transaction (create stay, assignment & set status 'OCCUPIED')	Atomically creates guest stay, assigns physical room, and marks room OCCUPIED.
7 [Success]	Database	FrontDeskService	Confirm check-in transaction committed	Database confirms transaction committed successfully.
8 [Success]	FrontDeskService	FrontDeskController	returnCheckInResponse(roomKey, stayDetails)	Returns check-in summary DTO with RFID room key payload.
9 [Success]	FrontDeskController	CheckInPage	returnSuccessResponse(200)	Responds with HTTP 200 OK status.
10 [Success]	CheckInPage	Receptionist	Display Check-in Success & Print RFID Key Card	Renders check-in success badge and triggers RFID key encoder.
11 [Failure]	FrontDeskService	FrontDeskController	returnCheckInError(400)	Returns error if booking is not found or no clean room is available.
12 [Failure]	FrontDeskController	CheckInPage	returnErrorResponse(400)	Responds with HTTP 400 Bad Request error.
13 [Failure]	CheckInPage	Receptionist	Display Error ("No	Displays descriptive

			clean room available for assignment")	error alert banner on check-in interface.

4.5 Ancillary Services & Billing Management

The Ancillary Services & Billing subsystem coordinates on-premise hotel service catalogs (dining, laundry, spa), room folio charge posting, voucher discount applications, multi-currency invoicing, and payment settlements (UC-31 through UC-40). This section details UC-35 (Order Hotel Service) and UC-37/39 (Folio Invoicing & Payment Settlement).

4.5.1 Screen Design

Figure 4-14 illustrates the hotel service catalog interface used for posting service orders to guest folios. Table 4-11 lists its controls.

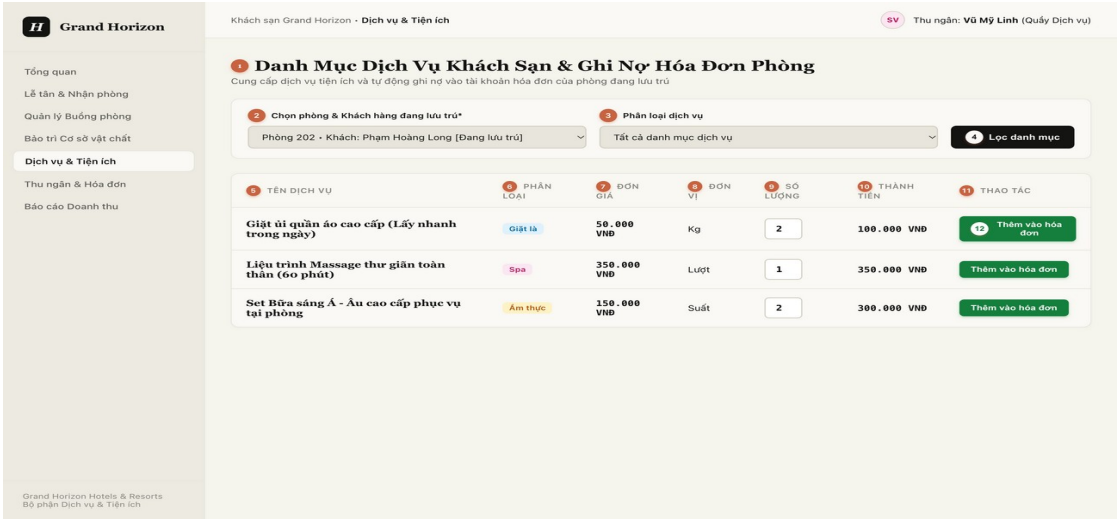


Figure 4-14: Screen Design of Available Hotel Services (UC-34 & UC-35)

Table 4-11: Screen Definition for Hotel Service Ordering (UC-35)

No	Object/Control Name	Type	Required	Length	Description
(1)	**lblCatalogTitle**	Label / Header	Yes	N/A	Header identifying the ancillary service catalog and folio debit workflow.
(2)	**cboActiveGuestStay**	Dropdown	Yes	N/A	Selects active occupied guest stay to receive service charges [DB Mapping: tbl_guest_stay.stay_id, tbl_room.room_number] (*).
(3)	**cboCategoryFilter**	Dropdown	No	N/A	Department categorization filter (All, Laundry, Spa & Wellness, Food & Beverage).

No	Object/Control Name	Type	Required	Length	Description
(4)	**btnFilterCatalog**	Button	No	Trigger API	Filters displayed service catalog list according to selected department.
(5)	**colServiceName**	Display / Text	Yes	100 chars	Commercial name of service item [DB Mapping: tbl_service.service_name].
(6)	**colCategory**	Display / Badge	Yes	50 chars	Department categorization badge (Laundry, Spa, Dining).
(7)	**colUnitPrice**	Display / Currency	Yes	N/A	Official standard unit price [DB Mapping: tbl_service.unit_price].
(8)	**colUnit**	Display / Text	Yes	20 chars	Service measurement unit (Kg, Session, Set).
(9)	**txtQuantity**	NumberInput	Yes	Min: 1, Max: 99	Order quantity selected by staff or guest [DB Mapping: tbl_service_order.quantity].
(10)	**colTotalAmount**	Display / Currency	Yes	N/A	Item charge subtotal dynamically computed as unit_price * quantity.
(11)	**colActionHeader**	Label / Header	Yes	N/A	Column header for row operational action buttons.
(12)	**btnAddToFolio**	Button	Yes	Trigger API	Direct order button [DB Action: INSERT into tbl_service_order (status='POSTED'); UPDATE tbl_folio.total_amount].

4.5.2 Class/Component Design

4.5.2.1 Backend Class Diagram

Figure 4-15 illustrates the backend class structure for Ancillary Services and Billing Management. It enforces Composition (*--) for Folio *-- Additional_Charge, Realization (..|>) for all billing and order services, Dependency (..>) for controller endpoints and JPA queries, and Association (-->) for repository references and billing entity associations.

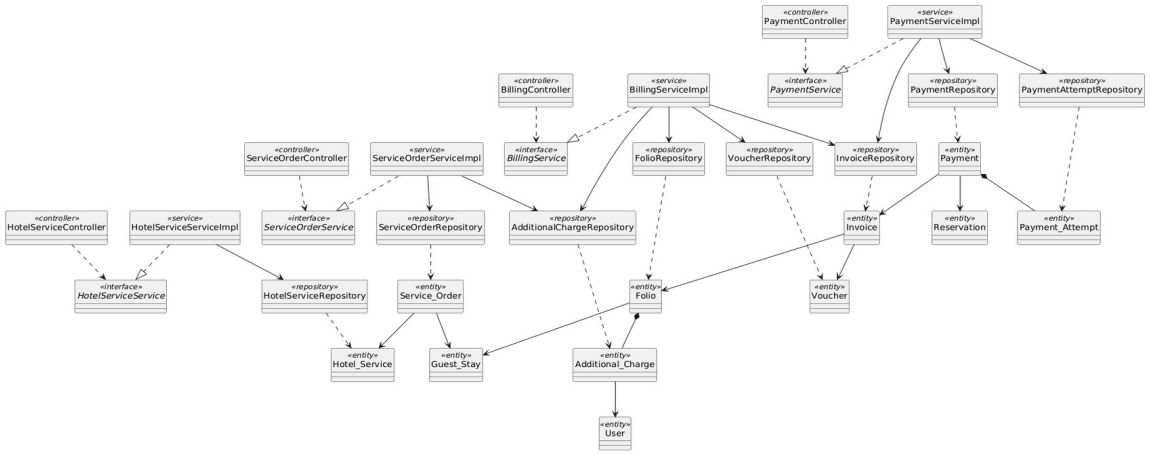


Figure 4-15: Backend Class/Component Structure Diagram for Ancillary Services & Billing

4.5.2.2 Classes/Component Description

4.5.2.2.1 BillingController Class

Class	**BillingController**			
Description	REST controller exposing endpoints for folio compilation, voucher discounts, checkout invoicing, and payment processing.			
Base Class	None			
Constructor	+ BillingController(billingService: BillingService)			
Prototype	@RestController @RequestMapping("/api/v1/billing") public class BillingController			
Source File	com/hotel/hms/controller/BillingController.java			
Namespace	com.hotel.hms.controller			
Attributes	**Name**	**Type**	**Description**	
	billingService	BillingService	Business logic interface for folio billing and payment settlements.	
Methods	**Name**	**Input**	**Output**	**Description**
	getFolioDetails	Long stayId	ResponseEntity<FolioDTO>	Retrieves itemized room and service charges for an active stay.
	processCheckOutPayment	PaymentSettlementDTO dto	ResponseEntity<InvoiceReceiptDTO>	Finalizes folio, records payment, and creates fiscal Invoice.

4.5.2.2.2 BillingServiceImpl Class

Class	BillingServiceImpl			
Description	Service executing folio item consolidation, tax/discount computations, payment gateway settlement, and invoice generation.			
Base Class	BillingService			
Constructor	+ BillingServiceImpl(folioRepo: FolioRepository, invoiceRepo: InvoiceRepository, paymentRepo: PaymentRepository, gateway: PaymentGateway)			
Prototype	@Service @Transactional public class BillingServiceImpl implements BillingService			
Source File	com/hotel/hms/service/impl/BillingServiceImpl.java			
Namespace	com.hotel.hms.service.impl			
Attributes	**Name**	**Type**	**Description**	
	folioRepository	FolioRepository	Access to guest folio records and items.	
	invoiceRepository	InvoiceRepository	Persistence for generated Invoice entities.	
	paymentRepository	PaymentRepository	Persistence for completed transaction payments.	
	paymentGateway	PaymentGateway	Integration client for electronic payment transactions.	
Methods	**Name**	**Input**	**Output**	**Description**
	settleFolioAndCheckout	PaymentSettlementDTO dto	InvoiceReceiptDTO	Validates balance, executes transaction, issues Invoice, and closes Folio.
	applyVoucher	Long folioId, String voucherCode	FolioDTO	Applies discount voucher to folio grand total.

4.5.2.2.3 Folio Class

Class	Folio
Description	JPA entity mapped to table 'Folio'. Tracks aggregate financial charges and balance incurred during a guest stay.
Base Class	None
Constructor	+ Folio()
Prototype	@Entity @Table(name = "Folio") public class Folio

Source File	com/hotel/hms/entity/Folio.java			
Namespace	com.hotel.hms.entity			
Attributes	**Name**	**Type**	**Description**	
	folioId	Long	Primary key identifier of folio.	
	createdAt	LocalDateTime	Timestamp when folio account was created upon check-in (mapped to Folio.created_at in Section 5).	
	totalAmount	BigDecimal	Current cumulative financial balance.	
	status	String	Folio status: OPEN, SETTLED, CLOSED.	
	guestStay	Guest_Stay	One-to-One reference to Guest_Stay entity.	
Methods	**Name**	**Input**	**Output**	**Description**
	getFolioId	void	Long	Returns folio primary key.
	getTotalAmount	void	BigDecimal	Returns current balance.

4.5.2.2.4 Invoice Class

Class	Invoice			
Description	JPA entity mapped to table 'Invoice'. Represents legally binding fiscal receipt issued upon stay settlement.			
Base Class	None			
Constructor	+ Invoice()			
Prototype	@Entity @Table(name = "Invoice") public class Invoice			
Source File	com/hotel/hms/entity/Invoice.java			
Namespace	com.hotel.hms.entity			
Attributes	**Name**	**Type**	**Description**	
	invoiceId	Long	Primary key identifier of invoice.	
	issueDate	LocalDateTime	Timestamp when invoice was generated upon checkout (mapped to Invoice.issue_date in Section 5).	
	totalAmount	BigDecimal	Final billed total including taxes and discounts.	

	finalAmount	BigDecimal	Payable total amount after voucher discount deduction (mapped to Invoice.final_amount in Section 5).	
	voucher	Voucher	Optional promotional discount voucher applied (Many-to-One, mapped to Invoice.voucher_id in Section 5).	
	status	String	Invoice settlement status: PAID, UNPAID, CANCELLED (mapped to Invoice.status in Section 5).	
	folio	Folio	Many-to-One reference to settled Folio.	
Methods	**Name**	**Input**	**Output**	**Description**
	getInvoiceId	void	Long	Returns invoice ID.
	getTotalAmount	void	BigDecimal	Returns grand total.

4.5.3 Object Interactions (Sequence Diagram)

Figure 4-16 illustrates the runtime interaction flow for UC-37 & UC-39 Check-out & Payment Settlement. Table 4-12 details each step.

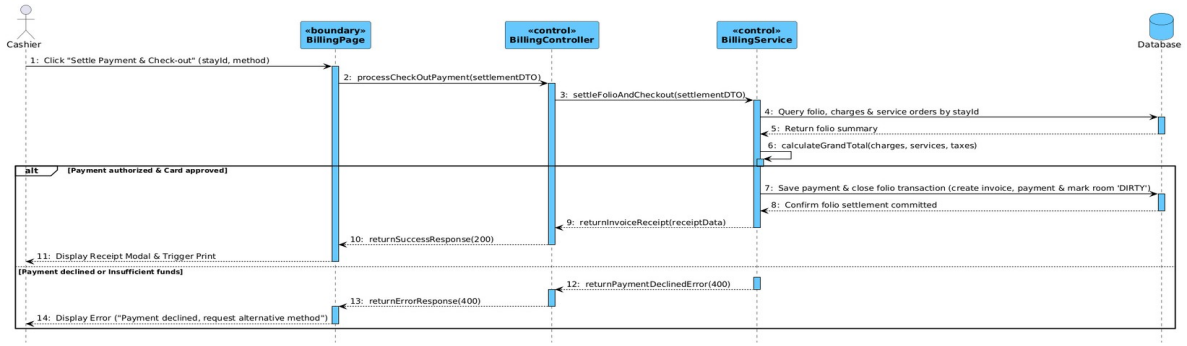


Figure 4-16: Sequence Diagram for UC-37 & UC-39 Check-out & Payment Settlement

Table 4-12: Interaction Steps for UC-37 & UC-39 Check-out & Payment Settlement

Step	Caller (From)	Callee (To)	Action / Method	Description
1	Cashier	BillingPage	Click "Settle Payment & Check-out" (stayId, method)	Cashier verifies folio charges and initiates final payment settlement.
2	BillingPage	BillingController	processCheckOutPayment(settlementDTO)	Dispatches HTTP POST request with settlement parameters.

3	BillingController	BillingService	settleFolioAndCheckout(settlementDTO)	Delegates folio closing and payment settlement to billing service.
4	BillingService	Database	Query folio, charges & service orders by stayId	Loads folio line items, room charges, and consumed service orders.
5	Database	BillingService	Return folio summary	Returns itemized consumption records and preliminary balance.
6	BillingService	BillingService	calculateGrandTotal(charges, services, taxes)	Calculates net subtotal, applicable taxes (VAT), discounts, and total due.
7 [Success]	BillingService	Database	Save payment & close folio transaction (create invoice, payment & mark room 'DIRTY')	Atomically creates fiscal invoice, records payment, closes folio, and sets room DIRTY.
8 [Success]	Database	BillingService	Confirm folio settlement committed	Database confirms payment and folio transaction committed.
9 [Success]	BillingService	BillingController	returnInvoiceReceipt(receiptData)	Returns itemized invoice receipt DTO.
10 [Success]	BillingController	BillingPage	returnSuccessResponse(200)	Responds with HTTP 200 OK status.
11 [Success]	BillingPage	Cashier	Display Receipt Modal & Trigger Print	Displays receipt modal with invoice breakdown and triggers print dialog.
12 [Failure]	BillingService	BillingController	returnPaymentDeclinedError(400)	Returns payment gateway rejection or insufficient funds error.
13 [Failure]	BillingController	BillingPage	returnErrorResponse(400)	Responds with HTTP 400 Bad Request error.
14 [Failure]	BillingPage	Cashier	Display Error ("Payment declined, request alternative method")	Displays payment failure notification requesting alternative payment.

4.6 Reviews, Audit & Reporting Management

The Reviews, Audit & Reporting subsystem oversees guest satisfaction feedback, automated audit logging of sensitive actions, and management operational intelligence reports covering room occupancy ratios, revenue per available room (RevPAR), and department revenue trends (UC-41 through UC-50). This section details UC-41 (Submit Room Review) and UC-46 (Revenue Analytics Report).

4.6.1 Screen Design

Figure 4-17 illustrates the executive revenue analytics dashboard (UC-46). Table 4-13 specifies the control definitions for the interface.

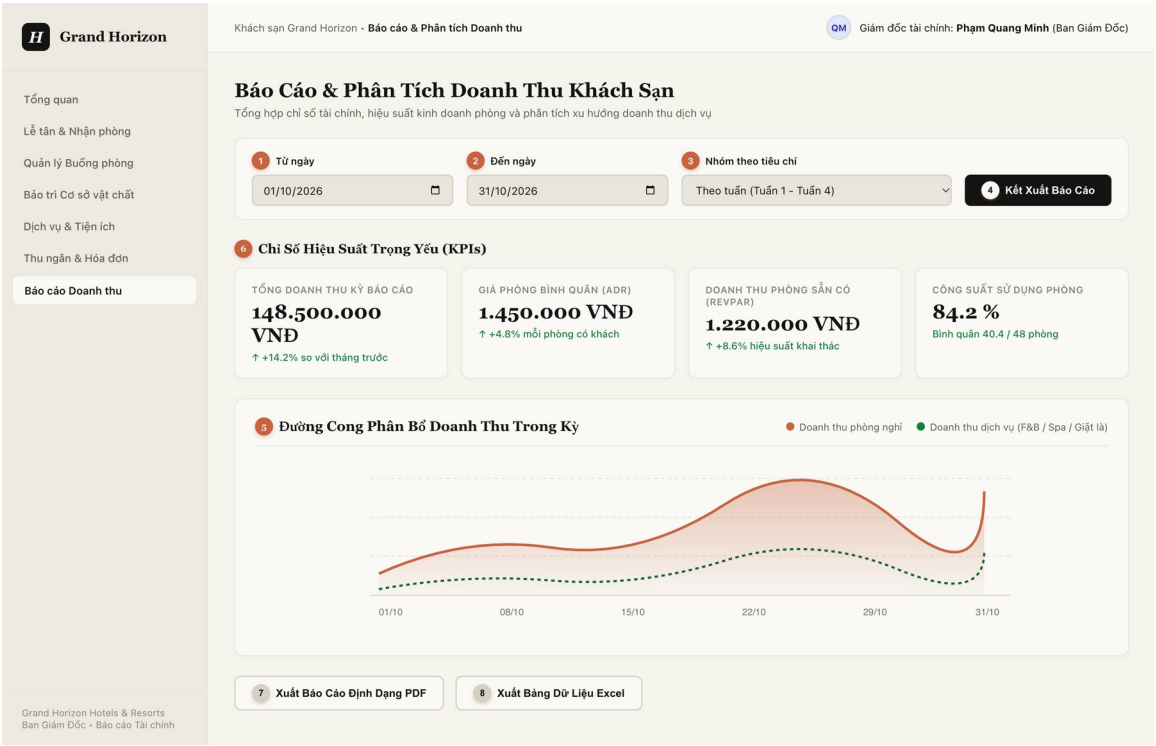


Figure 4-17: Screen Design of Management Revenue Analytics Dashboard (UC-46)

Table 4-13: Screen Definition for Management Revenue Analytics (UC-46)

No	Object/Control Name	Type	Required	Length	Description
(1)	**dtFromDate**	DatePicker	Yes	YYYY-MM-DD	Start date of analytics reporting period filter (*).
(2)	**dtToDate**	DatePicker	Yes	YYYY-MM-DD	End date of analytics reporting period filter (*).
(3)	**cboReportGrouping**	Dropdown	Yes	N/A	Temporal grouping criteria: Daily, Weekly, Monthly, or by Room Category.

No	Object/Control Name	Type	Required	Length	Description
(4)	**btnGenerateReport**	Button	Yes	Trigger API	Dispatches analytics query to backend; aggregates data from tbl_invoice, tbl_payment, and tbl_service_order.
(5)	**chartRevenueTrend**	Chart	Yes	Area / Line Chart	Renders graphical revenue curve comparing room tariff revenue vs ancillary service revenue.
(6)	**kpiCards**	KPICard / Display	Yes	Metrics	Executive summary metric cards: Gross Property Revenue, ADR, RevPAR, and Room Occupancy Rate.
(7)	**btnExportPDF**	Button	No	N/A	Exports current analytics view to structured executive PDF report.
(8)	**btnExportExcel**	Button	No	N/A	Exports raw transactional dataset to Microsoft Excel workbook.

4.6.2 Class/Component Design

4.6.2.1 Backend Class Diagram

Figure 4-18 illustrates the backend class structure for Reviews & Reporting. It uses flat layout with standard UML relationship arrows: Realization (..|>) for reporting services, Dependency (..>) for controller endpoints and query invocations, and Association (-->) for repository references and review entity associations.

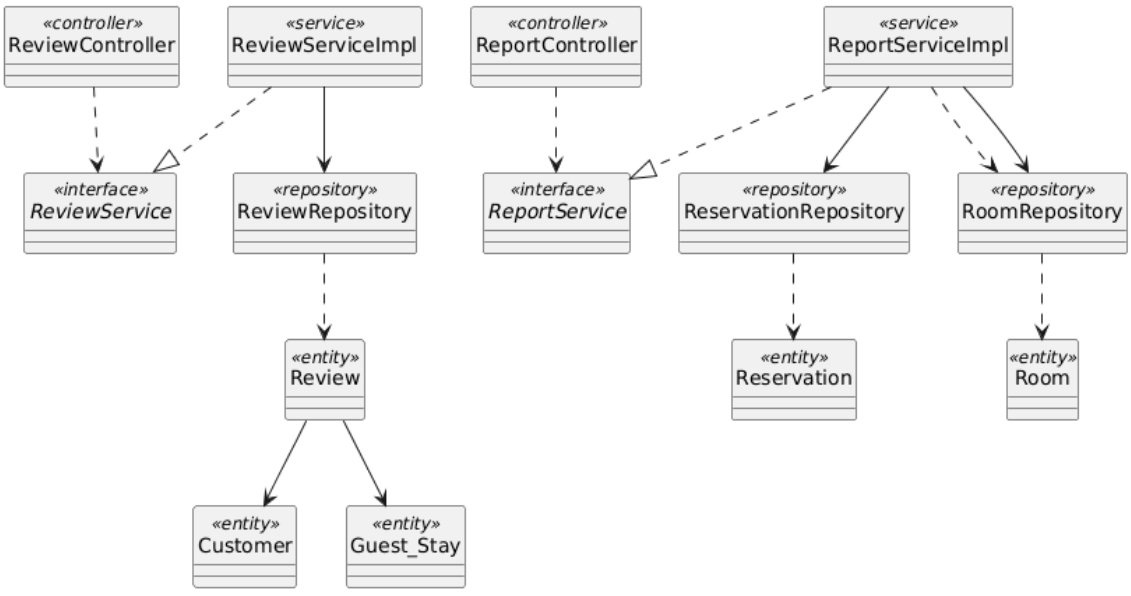


Figure 4-18: Backend Class/Component Structure Diagram for Reviews & Reporting

4.6.2.2 Classes/Component Description

4.6.2.2.1 ReportController Class

Class	ReportController			
Description	REST controller exposing analytical endpoints for room occupancy and property revenue intelligence.			
Base Class	None			
Constructor	+ ReportController(reportService: ReportService)			
Prototype	@RestController @RequestMapping("/api/v1/reports") public class ReportController			
Source File	com/hotel/hms/controller/ReportController.java			
Namespace	com.hotel.hms.controller			
Attributes	**Name**	**Type**	**Description**	
	reportService	ReportService	Service calculating analytical KPIs.	
Methods	**Name**	**Input**	**Output**	**Description**
	getRevenueReport	LocalDate from, LocalDate to	ResponseEntity<RevenueReportDTO>	Calculates total revenue, ADR, and RevPAR over time window.
	getOccupancyReport	LocalDate from, LocalDate to	ResponseEntity<OccupancyReportDTO>	Calculates room occupancy percentages and turnover ratios.

4.6.2.2.2 ReportServiceImpl Class

Class	ReportServiceImpl		
Description	Service aggregating multi-table relational data to produce executive business intelligence metrics.		
Base Class	ReportService		
Constructor	+ ReportServiceImpl(resRepo: ReservationRepository, roomRepo: RoomRepository)		
Prototype	@Service @Transactional(readOnly = true) public class ReportServiceImpl implements ReportService		
Source File	com/hotel/hms/service/impl/ReportServiceImpl.java		
Namespace	com.hotel.hms.service.impl		
Attributes	**Name**	**Type**	**Description**

	reservationRepository	ReservationRepository	Read-only access for booking revenue aggregation.	
	roomRepository	RoomRepository	Read-only access for total physical room capacity queries.	
Methods	**Name**	**Input**	**Output**	**Description**
	generateRevenueReport	LocalDate from, LocalDate to	RevenueReportDTO	Aggregates revenue, computes Average Daily Rate (ADR) and RevPAR.
	compileSummaryKPIs	BigDecimal rev, Integer rooms	KPIOverviewDTO	Synthesizes high-level summary cards.

4.6.2.2.3 ReviewRepository Class

Class	**ReviewRepository**			
Description	Spring Data JPA repository managing guest reviews and satisfaction ratings.			
Base Class	JpaRepository<Review, Long>			
Constructor	None (Spring Data JPA Interface)			
Prototype	@Repository public interface ReviewRepository extends JpaRepository<Review, Long>			
Source File	com/hotel/hms/repository/ReviewRepository.java			
Namespace	com.hotel.hms.repository			
Attributes	**Name**	**Type**	**Description**	
	None	-	Inherits Spring Data JPA persistence context.	
Methods	**Name**	**Input**	**Output**	**Description**
	findByCustomer_CustomerId	Long customerId	List<Review>	Queries reviews submitted by customer.
	calculateAverageRating	void	Double	Calculates average property rating score across all reviews.

4.6.2.2.4 Review Class

Class	**Review**
Description	JPA entity mapped to table 'Review'. Stores star rating, feedback comment, and submission date.
Base Class	None

Constructor	+ Review()			
Prototype	@Entity @Table(name = "Review") public class Review			
Source File	com/hotel/hms/entity/Review.java			
Namespace	com.hotel.hms.entity			
Attributes	**Name**	**Type**	**Description**	
	reviewId	Long	Primary key identifier of review.	
	rating	Integer	Rating score from 1 to 5 stars.	
	comment	String	Written feedback notes from guest.	
	reviewDate	LocalDateTime	Timestamp when review was submitted.	
	customer	Customer	Many-to-One reference to Customer entity.	
	guestStay	Guest_Stay	Many-to-One reference to completed hotel stay (mapped to Review.stay_id in Section 5).	
Methods	**Name**	**Input**	**Output**	**Description**
	getRating	void	Integer	Returns star rating.
	getComment	void	String	Returns written feedback.

4.6.3 Object Interactions (Sequence Diagram)

Figure 4-19 depicts the runtime sequence flow for UC-46 Generate Revenue Analytics Report. Table 4-14 details each step.

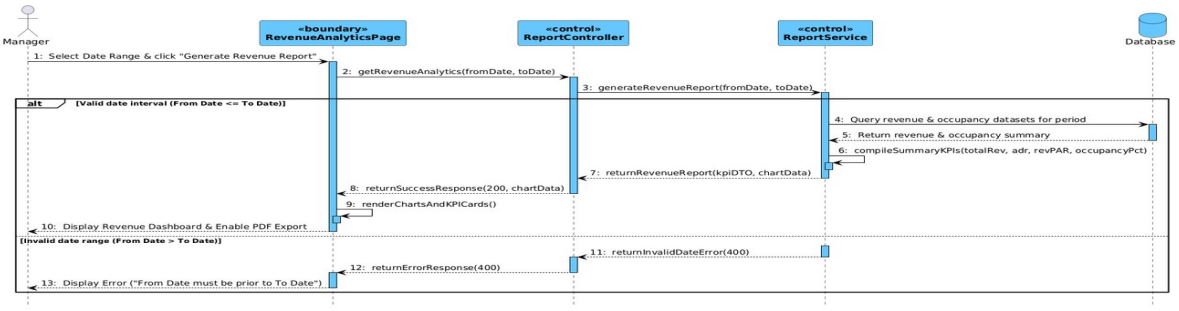


Figure 4-19: Sequence Diagram for UC-46 Generate Revenue Analytics Report

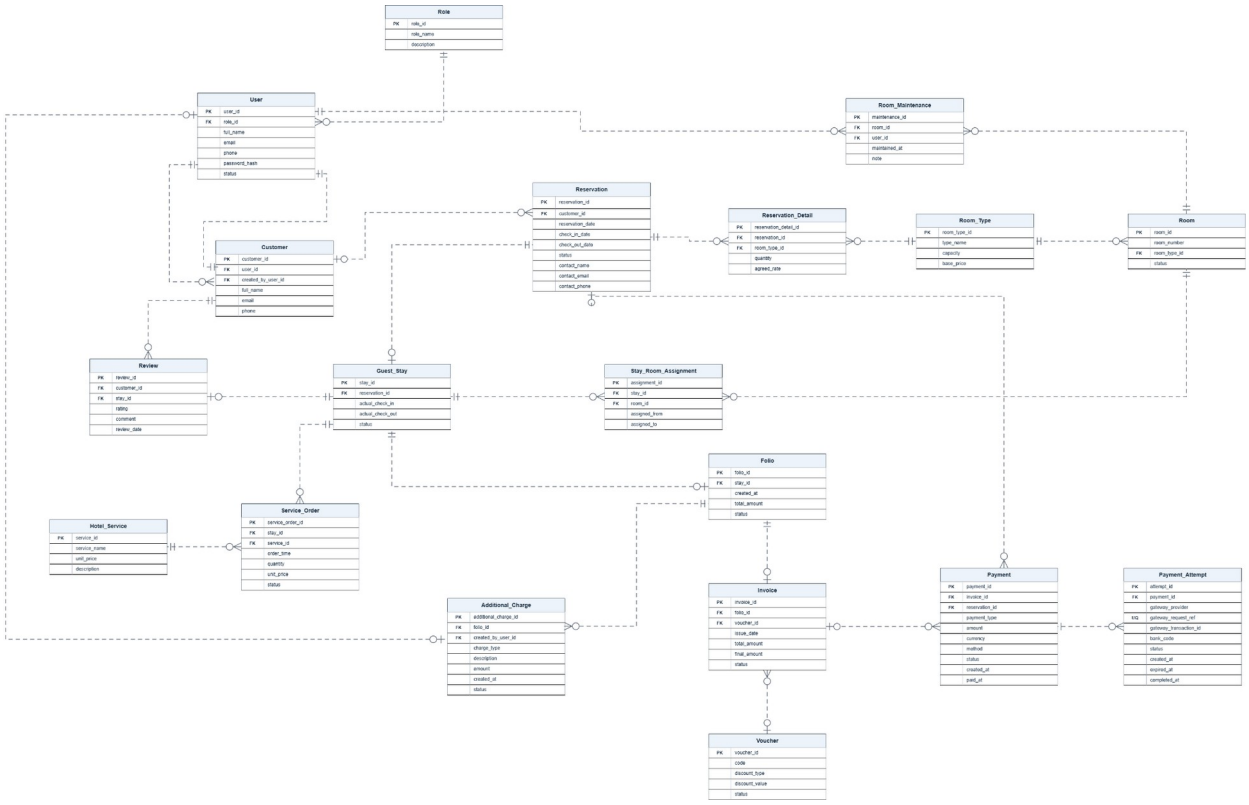
Table 4-14: Interaction Steps for UC-46 Generate Revenue Analytics Report

Step	Caller (From)	Callee (To)	Action / Method	Description
1	Manager	RevenueAnalyticsPage	Select Date Range & click "Generate Revenue Report"	Hotel Manager selects date interval and requests revenue report.
2	RevenueAnalyticsPage	ReportController	getRevenueAnalytics(fromDate, toDate)	Dispatches HTTP GET request with date filter query parameters.
3	ReportController	ReportService	generateRevenueReport(fromDate, toDate)	Delegates analytical reporting calculations to reporting service.
4 [Success]	ReportService	Database	Query revenue & occupancy datasets for period	Queries invoices, room revenue, and daily occupied room inventory.
5 [Success]	Database	ReportService	Return revenue & occupancy summary	Returns aggregate metrics dataset for requested period.
6 [Success]	ReportService	ReportService	compileSummaryKPIs(totalRev, adr, revPAR, occupancyPct)	Calculates key KPIs: Total Revenue, ADR, RevPAR, and Occupancy Rate.
7 [Success]	ReportService	ReportController	returnRevenueReport(kpiDTO, chartData)	Returns compiled report DTO with time-series chart data.
8 [Success]	ReportController	RevenueAnalyticsPage	returnSuccessResponse(200, chartData)	Responds with HTTP 200 OK and chart dataset.
9 [Success]	RevenueAnalyticsPage	RevenueAnalyticsPage	renderChartsAndKPICards()	Renders revenue trend line charts and summary KPI cards.
10 [Success]	RevenueAnalyticsPage	Manager	Display Revenue Dashboard & Enable PDF Export	Displays analytics dashboard and activates PDF/Excel export buttons.
11 [Failure]	ReportService	ReportController	returnInvalidDateError(400)	Returns date validation error when From Date is after To Date.
12 [Failure]	ReportController	RevenueAnalyticsPage	returnErrorResponse(400)	Responds with HTTP 400 Bad Request error.
13 [Failure]	RevenueAnalyticsPage	Manager	Display Error ("From	Displays date range

			Date must be prior to To Date")	error toast message to Manager.

5. Database Design

5.1 Database Design



a. Role

Stores the system roles used for access control and user permissions.

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	role_id	INT	4	Yes	Yes	PK	
2	role_name	NVARCHAR	100	Yes	Yes		
3	description	NVARCHAR	500	No	No		

b. User

Stores accounts for customers and hotel staff, including login credentials and access roles.

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	user_id	INT	4	Yes	Yes	PK	
2	role_id	INT	4	No	Yes	FK	References Role.role_id.
3	full_name	NVARCHAR	150	No	Yes		
4	email	VARCHAR	254	Yes	Yes		
5	phone	VARCHAR	20	No	No		
6	password_hash	VARCHAR	255	No	Yes		Hashed password only; never store plaintext.
7	status	VARCHAR	30	No	Yes		

c. Room_Type

Defines room categories, guest capacity, and the standard rate for each category.

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	room_type_id	INT	4	Yes	Yes	PK	
2	type_name	NVARCHAR	100	Yes	Yes		
3	capacity	INT	4	No	Yes		
4	base_price	DECIMAL	18,2	No	Yes		

d. Room

Represents a physical hotel room belonging to one room type.

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	room_id	INT	4	Yes	Yes	PK	
2	room_number	VARCHAR	30	Yes	Yes		
3	room_type_id	INT	4	No	Yes	FK	References

							Room_Type.room_type_id.
4	status	VARCHAR	30	No	Yes		

e. Reservation

Records hotel reservations, including the customer responsible for the booking, planned stay dates, reservation status, and contact information.

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	reservation_id	INT	4	Yes	Yes	PK	
2	customer_id	INT	4	No	Yes	FK	References Customer.customer_id .
3	reservation_date	DATETIME 2	8	No	Yes		
4	check_in_date	DATE	3	No	Yes		
5	check_out_date	DATE	3	No	Yes		
6	status	VARCHAR	30	No	Yes		
7	contact_name	NVARCHAR	150	No	Yes		
8	contact_email	VARCHAR	254	No	No		
9	contact_phone	VARCHAR	20	No	Yes		

f. Reservation_Detail

Records the room types, quantities, and agreed rates requested within each reservation.

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	reservation_detail_id	INT	4	Yes	Yes	PK	
2	reservation_id	INT	4	No	Yes	FK	References Reservation.reservation_id.
3	room_type_id	INT	4	No	Yes	FK	References Room_Type.room_type_id.
4	quantity	INT	4	No	Yes		
5	agreed_rate	DECIMAL	18,2	No	Yes		

g. Guest_Stay

Records an actual hotel stay resulting from a reservation.

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	stay_id	INT	4	Yes	Yes	PK	
2	reservation_id	INT	4	Yes	Yes	FK	References Reservation.reservation_id.
3	actual_check_in	DATETIME2	8	No	Yes		
4	actual_check_out	DATETIME2	8	No	No		
5	status	VARCHAR	30	No	Yes		

h. Stay_Room_Assignment

Records the physical rooms assigned to a stay and the assignment periods.

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	assignment_id	INT	4	Yes	Yes	PK	
2	stay_id	INT	4	No	Yes	FK	References Guest_Stay.stay_id.
3	room_id	INT	4	No	Yes	FK	References Room.room_id.
4	assigned_from	DATETIME2	8	No	Yes		
5	assigned_to	DATETIME2	8	No	No		

i. Room_Maintenance

Records room maintenance activities performed or logged by staff.

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	maintenance_id	INT	4	Yes	Yes	PK	
2	room_id	INT	4	No	Yes	FK	References Room.room_id.

3	user_id	INT	4	No	Yes	FK	References User.user_id.
4	maintained_at	DATETIME2	8	No	Yes		
5	note	NVARCHAR	500	No	No		

j. Review

Stores customer feedback associated with a completed stay.

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	review_id	INT	4	Yes	Yes	PK	
2	customer_id	INT	4	No	Yes	FK	References Customer.customer_id.
3	stay_id	INT	4	No	Yes	FK	References Guest_Stay.stay_id.
4	rating	INT	4	No	Yes		
5	comment	NVARCHAR	500	No	No		
6	review_date	DATETIME2	8	No	Yes		

k. Hotel_Service

Defines additional hotel services and their standard prices.

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	service_id	INT	4	Yes	Yes	PK	
2	service_name	NVARCHAR	100	No	Yes		
3	unit_price	DECIMAL	18,2	No	Yes		
4	description	NVARCHAR	500	No	No		

l. Service_Order

Records services ordered during an active stay.

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	service_order_id	INT	4	Yes	Yes	PK	

2	stay_id	INT	4	No	Yes	FK	References Guest_Stay.stay_id.
3	service_id	INT	4	No	Yes	FK	References Hotel_Service.service_id.
4	order_time	DATETIME 2	8	No	Yes		
5	quantity	INT	4	No	Yes		
6	unit_price	DECIMAL	18,2	No	Yes		
7	status	VARCHAR	30	No	Yes		

m. Folio

Maintains the running account of charges associated with a hotel stay.

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	folio_id	INT	4	Yes	Yes	PK	
2	stay_id	INT	4	Yes	Yes	FK	References Guest_Stay.stay_id.
3	created_at	DATETIME2	8	No	Yes		
4	total_amount	DECIMAL	18,2	No	Yes		
5	status	VARCHAR	30	No	Yes		

n. Additional_Charge

Records additional charges applied to a guest's folio, such as property damage fees, late check-out fees, and other surcharges. Charges can be voided without deleting their history.

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	additional_charge_id	INT	4	Yes	Yes	PK	
2	folio_id	INT	4	No	Yes	FK	References Folio.folio_id.

3	created_by_user_id	INT	4	No	Yes	FK	References User.user_id.
4	charge_type	VARCHAR	30	No	Yes		DAMAGE, LATE_CHECKOUT, OTHER.
5	description	NVARCHAR	500	No	Yes		Reason for the additional charge.
6	amount	DECIMAL	18,2	No	Yes		Charge amount.
7	created_at	DATETIME2	8	No	Yes		
8	status	VARCHAR	30	No	Yes		ACTIVE, VOIDED.

o. Voucher

Stores promotional vouchers and their discount rules.

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	voucher_id	INT	4	Yes	Yes	PK	
2	code	VARCHAR	30	Yes	Yes		
3	discount_type	VARCHAR	30	No	Yes		
4	discount_value	DECIMAL	18,2	No	Yes		
5	status	VARCHAR	30	No	Yes		

p. Invoice

Stores checkout billing information generated from a folio.

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	invoice_id	INT	4	Yes	Yes	PK	
2	folio_id	INT	4	Yes	Yes	FK	References Folio.folio_id.
3	voucher_id	INT	4	No	No	FK	References Voucher.voucher_id.
4	issue_date	DATETIME2	8	No	Yes		
5	total_amount	DECIMAL	18,2	No	Yes		
6	final_amount	DECIMAL	18,2	No	Yes		
7	status	VARCHAR	30	No	Yes		

q. Payment

Stores payments associated with either a reservation or an invoice.

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	payment_id	INT	4	Yes	Yes	PK	
2	invoice_id	INT	4	No	No	FK	References Invoice.invoice_id. Exactly one of these two references should be populated.
3	reservation_id	INT	4	No	No	FK	References Reservation.reservation_id. Exactly one of these two references should be populated.
4	payment_type	VARCHAR	30	No	Yes		
5	currency	CHAR	3	No	Yes		Currency code, e.g. VND
6	amount	DECIMAL	18,2	No	Yes		
7	method	VARCHAR	30	No	Yes		

8	status	VARCHAR	30	No	Yes		
9	created_at	DATETIME 2	8	No	Yes		Time the payment obligation was created.
10	paid_at	DATETIME 2	8	No	No		Time the payment was confirmed as paid.

r. Customer

Stores guest/customer profiles used for reservations and reviews. A profile may exist without a login account.

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes
1	customer_id	INT	4	Yes	Yes	PK	
2	user_id	INT	4	Yes	No	FK	References User.user_id. Optional link to customer login; one customer profile per user.
3	created_by_user_id	INT	4	No	No	FK	References User.user_id.
4	full_name	NVARCHAR	150	No	Yes		
5	email	VARCHAR	254	No	No		
6	phone	VARCHAR	20	No	Yes		

s. Payment_Attempt

Records each payment attempt made through an online payment gateway, including the gateway request reference, transaction identifier, processing status, and relevant timestamps.

#	Field name	Type	Size	Unique	Not Null	PK/FK	Notes

1	attempt_id	INT	4	Yes	Yes	PK	
2	payment_id	INT	4	No	Yes	FK	References Payment.payment_id.
3	gateway_provider	VARCHAR	30	No	Yes		E.g. VNPAY.
4	gateway_request_ref	VARCHAR	100	Yes	Yes		Maps to vnp_TxnRef.
5	gateway_transaction_id	VARCHAR	100	No	No		Maps to vnp_TransactionNo.
6	bank_code	VARCHAR	30	No	No		Maps to vnp_BankCode.
7	status	VARCHAR	30	No	Yes		PENDING, SUCCESS, FAILED, EXPIRED.
8	created_at	DATETIME2	8	No	Yes		Time the attempt was created.
9	expired_at	DATETIME	8	No	No		Payment request

		2					expiration time.
10	completed_at	DATETIME 2	8	No	No		Time the attempt reached a final state.

5.2 Data File Design

[File List, in the table format as below]

#	File Name	Type	Notes
1	Class name1		
2	Class name2		
3	..		